TECHNISCHE HOCHSCHULE DEGGENDORF
Campus Cham

Master's Thesis

Deggendorf Institute of Technology, Campus Cham
Faculty of Applied Natural Sciences and Industrial Engineering
Master Mechatronics & Cyber-Physical Systems

Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

Master's thesis to obtain the academic degree:
Master of Engineering (M.Eng.)

Submitted by: Sooraj Sudhakaran.
Matriculation Number: XXXXXXXX
First Examiner: Prof. Dr. Matthias Górka

Cham, March 23, 2026

Abstract

Acknowledgements

List of Tables

3.1 Functional Requirements for Automatic Robot Program Generation 13
3.2 Non-Functional Requirements — NFR-01 to NFR-04 with acceptance criteria and verification methods 15
3.3 Requirement Traceability Matrix — all fifteen requirements (FR-01 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios 22

4.1 Structural comparison of Concept A, Concept B, and Concept C across key design features 31
4.2 Structural comparison of Concept A, Concept B, and Concept C across key design features 34

5.1 Software modules across system layers and their primary responsibilities 41

List of Figures

3.1 Tool Center Point parameters in Drag&Bot. 20

5.1 System architecture 38
5.2 Five layer architecture 40
5.3 Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [11]). 44
5.4 Robot control panel interface used for system operation and monitoring. 46
5.5 Virtual simulation environment used for implementation and validation. The setup includes the robot, press brake, orientation table, and sheet workpiece. Source: own work. 48
5.6 Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work. 49
5.7 Operator Workflow .Every gate must be passed before the next begins.All operator interactions occu within the drag and bot enviornment. *Source: own work.* 50
5.8 TCP of UR5e robot. *Source: own work.* 52
5.9 Zone classification for robot workspace segmentation. *Source: own work.* 53
5.10 Fixture pose deviation validation workflow. Each monitored fixture is evaluated by comparing positional and angular deviations against defined thresholds. Out-of-tolerance or missing fixtures trigger a blocking warning dialog requiring operator confirmation. Validation is applied uniformly across all spatial zones. Source: own work. 55
5.11 Three-level system architecture illustrating the hierarchical structure of the implementation. *Source: own work.* 59

List of Abbreviations

OLP	:	Offline Programming
ROS	:	Robot Operating System
TCP	:	Tool Center Point
UI	:	User Interface
CAD	:	Computer-Aided Design
CAM	:	Computer-Aided Manufacturing
API	:	Application Programming Interface
DOF	:	Degree of Freedom
FR	:	Functional Requirements
NFR	:	Non-Functional Requirements
WSM	:	Weighted Sum Model
URDF	:	Unified Robot Description Format
YAML	:	YAML Ain’t Markup Language
Digital IO	:	Digital Input/Output
JSON	:	JavaScript Object Notation
SHA-512	:	Secure Hash Algorithm 512-bit
X	:	Translation along the X-axis (metres)
Y	:	Translation along the Y-axis (metres)
Z	:	Translation along the Z-axis (metres)
RX	:	Rotation about the X-axis (degrees)
RY	:	Rotation about the Y-axis (degrees)
RZ	:	Rotation about the Z-axis (degrees)
WebSocket	:	Web-based communication protocol for real-time data exchange defined in RFC 6455
LIN	:	Linear motion command
PTP	:	Point-to-point motion command
HI Dialog	:	Human Interaction (Drag&Bot-specific dialog service)
HTTP	:	Hypertext Transfer Protocol
HTTPS	:	Hypertext Transfer Protocol Secure
IP	:	Internet Protocol
DNB	:	Drag&Bot
UR5e	:	Universal Robots 5e (robot model)

Contents

Abstract **I**

Acknowledgements **III**

List of Tables **V**

List of Figures **VII**

List of Abbreviations **IX**

1 Introduction **1**
1.1 head of robot 1

2 State of the Art **2**
2.1 Manual And teach pendant based robot programming. 2
2.2 Offline programming approaches for industrial robots 3
2.3 Automatic and Rule-Based Robot Program Generation 4
2.4 Robotic Sheet-Metal Bending and Press-Brake Cells 5
2.5 Summary of the State of the Art and Identified Research Gap . 7

3 System Concept and Methodology **9**
3.1 Problem Definition and Objectives 9
3.2 Requirement Definition and Management 10
3.2.1 Functional Requirements 11
3.2.2 Non-functional Requirements 12
3.2.3 System Constraints and Assumptions 14
3.2.4 Software Platform and Tool requirement 17
3.2.4.1 SR-01 Drag And Bot runtime Platform 17
3.2.4.2 SR-02 Anybot Interface as Exclusive Communication layer. 18
3.2.4.3 SR-03 Tool Centre point. 19
3.2.5 System Traceability 20

4 Development of Concepts **23**
4.1 Purpose and scope of concept development 23
4.2 Concept A – Structured operator Workflow with Guided Teach in programming 24

4.2.1 Methodological Framework 24
4.2.2 System Structure 25
4.2.3 Critical Evaluation 25
4.3 Concept B – Automatic Program Generation based on rules with Integrated Constraint Validation. 26
4.3.1 Methodological Framework 26
4.3.2 system structure 28
4.3.3 Critical evaluation 28
4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation 29
4.4.1 Methodological Framework 29
4.4.2 System Structure 29
4.4.3 Critical Evaluation 30
4.5 Structural Comparison of Concepts 30
4.6 Concept Decision-Weighed Sum Model (WSM) 32
4.6.1 Methodology 32
4.6.2 Evaluation Criteria and Weights 32
4.7 Result of the weighted Sum Model 33
4.8 Analysis of results and Concept Decision 33

5 Implementation 37
5.1 Introduction and scope 37
5.2 System architecture 39
5.2.1 Layered Architecture Design and data flow 39
5.2.2 Module Overview and dependency Discipline 41
5.2.3 Communication Stack, Authentication interface Specification 43
5.2.4 Configuration Architecture and YAML File Structure 46
5.2.5 Operator interface – interactive 3d sheet bending viewer 47
5.3 Development and simulation Environment 50
5.3.1 Simulation Platform, Robot Model, and TCP Configuration 51
5.3.2 Virtual Cell Layout and Spatial Zone Classification 52
5.4 Pre-Execution Safety Validation 54
5.4.1 Fixture Pose Deviation Validation 54
5.4.2 Operator Authorization and Runtime Error Monitoring . . 56
5.5 Workpiece Orientation Strategy 56
5.5.1 Zone-Based RZ Admissibility 57
5.5.2 Pickup Orientation Validation 57
5.5.3 Three-Level Orientation Selection Strategy 57

References **61**

1 Introduction

YSBASHFKJSDHFKfdghghjg

1.1 head of robot

XGFGF sooraj sudhakaran

2 State of the Art

2.1 Manual And teach pendant based robot programming.

The programming of the industrial robots follows a programming approach based on teach pendant programming, which means, the robot movement and motions are explicitly defined and programmed directly during the operations. The programmer subsequently uses this approach to position and then programs the robot to move to a series of specific target positions. The robot subsequently saves these positions using programming languages specific to each robot type and brand. Despite advances in programming approaches for robotics systems, this method is still commonly used to its owing to its reliability and its usefulness in interacting with real world systems [1] . However, this may require expert knowledge of the robot's kinematics, tooling, and process constraints. Minor changes in geometry, fixture placement or cell layout often involve extensive manual reprogramming, increases commissioning effort and system downtime. Therefore, teach pendant programming scales poorly in flexible manufacturing systems characterized by frequent changeovers coupled with small batch sizes. In terms of the sheet metal handling and the automation of the press brake operations, the restrictions can appear even more considerable. When working with these pieces of sheet metal that can be flexible and large in size, their handling involves further challenges that need to be addressed, revolving around reachability issues and collision avoidance. In terms of manual programming , each step of the planned process of the bending involves the precise determination by the user of the intermediary positions for each action of the manipulator necessary for the accomplishment of the task at hand .Such an approach can appear heavily dependent on the users expertise for the purpose of avoiding the iterative try and error approach that might be inevitably required during the plant operations [2] .Especially , more classical robot programming paradigms have be identified as insufficient in terms of abstraction and flexibility needs for current applications in industrial robotics, especially when high flexibility and fast reconfiguration are at stake [1] .Such an approach to programming the robot , through interaction with the teach pendant itself , tend to create program that are more dependent on the specific type of the robot used and also the cell configuration, programming style of each particular operator etc.

As a result, manual teach in pendant programming methods have traditionally been seen to be unsuitable for advanced automation tasks. that often requires increased flexibility, analyses of process knowledge, and minimization of human interventions. It is realization that sparked off interest in alternative offline programming, task level programming and even methods of automatically generating robot programs, as discussed in the sections [1]

2.2 Offline programming approaches for industrial robots

The disadvantages associated with manual programming of industrial robotic arms lead to an increase in the uptake of offline programming in industrial applications. This programming method led to running the digital working environment using digital models of robots, tools, grippers and other components used for the programming without interrupting the real production. [1]

This programming is based on Computer aided Model (CAM) also known as the geometric or CAD models, which represent the robots, workpiece and all other components. These models allow robot programs to be simulated before they implement in the real working environment. Despite being associated with the disadvantages, offline programming also has some limitations when used in industrial applications.[3]

In robot- assisted sheet Metal handling presses brake operation, offline programming is commonly used for robot programs such as sheet metal handling, bending process, pick and regrip operations for sheet in different bending stages. Using virtual simulation, engineers can evaluate, validate and verify , and can identify the if the robot's reachability, regripping positions and bending sequences while safely avoiding the unreachable path to machines and fixtures. [2]. Even though this program offers many benefits, it relies heavily on exact digital models of robot cells. If the digital model does not perfectly match the real model configurations, due to the calibration error, fixture errors or in change in tool geometry model, the robot may misbehave in kinematics. In such cases, manual re- configuration needed, engineers must re-program during the commissioning. Another limitation of this programming is that it is less suitable for the environment with frequent changeovers and high-end product variety [1] This occurs because the programs are usually created for specific tasks and robot paths for, specific part geometry and bending process. [3] In case multi side bending process, regripping and regrasping operation is required, often require the workpiece to be temporarily place in orientation table and then grasped again in a dif-

ferent configuration. This is necessary to avoid collision with and optimize the robot kinematics. In this programming, these approaches are checked by the programmer, which increases the engineering effort that leads to limiting flexibility and time consuming. [2].

In summary, this program provides clear advantages and over manual programming by reducing the on-site teaching time and enabling virtual validation robot waypoints. However, it depends heavily on exact digital model and manual definition of task sequences reduces the efficiencies in industrial applications. These limitations have motivated into high level research, task oriented automatic program generation strategies, which aim to reduce human involvement while increasing the adaptability of the system. These approaches are explained in the following sections.

2.3 Automatic and Rule-Based Robot Program Generation

To reduce further engineering effort associated with manual and offline programming such as manual programming and offline programming, research in industrial robot section has increasingly focused on automatic program generation. This in-depth research aims to derive executable high level automated tasks with respect to robotics position and geometric information, thereby minimizing direct human re programming. In contrast to traditional offline programming, where engineer defines the motion trajectories, automated programming seeks to capture the environmental details in reusable and systematic representations.[1], [3].

A prominent solution executed in this method is rule based and task level robot program generation. In such methods, predefined rules or tasks are used to resolve the high-level constraints related to kinematics, tool orientation, and process feasibility. This rule-based programming reduces the operator role and enable the robot to perform intelligently for recurring task in industrial environment [3], [4]. Several researchers have been used reusable primitives as a foundation for automatic robot programming. Rather than programming individual motions, these complex robot behaviors represent a meaningful action such as grasping, repicking, or approaching target positions. [4], [5]. Automated programs is supported by geometric and feature based representation of parts such CAD Models or simplified geometric descriptions used for detecting or identifying the features such as grasping edges for the mechanical operations, approach directions, task sequences can be processed automatically. As compared to manual and conventional programming methods, this method enables adaptations to complex prod-

uct variants. [6]. This automated program generation has also been invented for sheet metal handling and robotic press brake automation applications, where repetitive task for bending operations and geometry is usually structured. In such systems, bending sequences, grasp configurations and placement strategies are derived from the constraints and geometric models rather than manual teaching. This reduces the engineer's effort and gives highly efficient output for repeatable similar workpieces. [2], [7].

Despite these advantages, rule-based robot program generations lead to some limitations. Most systems relay simply on the assumptions regarding part geometry, fixture layouts, tooling position and trajectories generations, which lead to restriction on applicability in highly variant part processes and low volume production scenarios. As task complexity increases, the number of rules may also get increased, which makes it hard to maintain the system, difficult to scale and lead to inconsistencies. [1], [4]. Regripping operations is one of the most highlighting problems in the sheet metal handling process. For multi-step bending , robot need to place the workpiece on temporary place called orientation table , which is used for the repegging and repositioning the workpiece for next sequence bending .Although there exist some regripping strategies , these are particularly for some specific tools and specific robot cells. [2], [7]. As a result, they do not support other setups well and limit the chances of automation.

Automatic program generation has also been combined with optimization-based methods, where trajectory quality is considered during motion generation. Such methods improve the execution performance with generating trajectory plans in robot handling tasks [8].

In conclusion, automatic and rule-based programs subsequently reduce the engineer's effort and increase consistency for complex task specifications. However, some approaches are still considered as the limitations, limited adaptability to process different variations about geometry and handling. These limitations give motivation to develop specific solutions with high degree automation, specifically for the sheet metal bending, which is the focus of this research.

2.4 Robotic Sheet-Metal Bending and Press-Brake Cells

Robotic press brake sheet metal bending has gained attention in industry due to the increasing demand for high productivity and consistent bending quality. In such systems, industrial robots are capable to handle sheet metal workpieces throughout the bending process, including gripping, positioning

at the press brake for bending process, reorientation for subsequent bending sequences and unloading process after the process completion. Compared to manual bending, robotic system reduces the operator's effort while ensures precise bending quality in a repeatable and reliable way. [1] Early research focused on analyzing feasibility of integrating industrial robots with traditional press brakes systems. These studies demonstrate that the sheet metal bending can be effectively performed when robot waypoints are carefully coordinated with the bending operation sequences. These results highlighted the importance of reliable synchronization between the robot and the press brake, particularly for precise process execution and accurate motion planning. [2]

Later studies of robotic sheet metal bending processes by researchers addressed the problem of process plaining, including the determination of bending sequences, selection of which tools need to be used for the specific bend and handling strategies for the workpiece. As a result, they showed bending depends on the geometry, reachability to process the bends. So, they found out that, make planning to make aware of unstable grasps or robot motions that cannot be executed during the process. [7] Unlike the rigid workpiece, the sheet metal continuously changes the shapes after each bending operation. This requires the robot to adjust the gripping position and adopt different orientational pickup through the process. To ensure safety, many robotic systems rely on conservative motion planning and fixed handling strategies. While this approach reduces the system complexity when dealing with the complex or varying part geometry. [1], [2] On the next stage, regripping becomes one of the most challenging parts for robotic bending. Often the sheet must be temporarily placed at an intermediate location and then repick again for the next bend, requiring appropriate regripping positions that are compatible with the bending tools. most existing systems rely on predefined regripping locations and manually planned regripping strategies. This significantly increases engineering effort and make system adaptability difficult for frequent process changes . [2], [7].

Research on path planning for robot assisted sheet metal bending shows constraints on their trajectories around press brake and tooling represents a major challenge. The waypoints must be carefully review in between the workspace especially during the reorientation and regripping operations [9].

Additional studies highlighted the role of grasp feasibility and stability, especially when vacuum based gripper are used. Research on vacuum gripper modelling and automatic grasp positions strongly depend on robustness and efficiency. [10]. However, these approaches tailored to specific de-

signed gripper types or handling strategies, and they are integrated into an automatic program generations press brake industrial applications

In industrial practice, robotic press brake bending performs well when the part geometry, tooling and process parameters are predefined. Such system typically relies on expensive offline preparation, accurate digital model simulations, manually defined handling and regripping strategies. However, as product variety increases, system flexibility significantly decreases. Consequently, the robotic press brake bending is well suited for structured and repetitive process. Nevertheless, current system still depends heavily on predefined process knowledge, pre planned trajectories and manually configured regripping strategies. These limitations reduce adaptability and have motivated extensive research into automated robot program generation methods that can respond to changing processes and production conditions. [1], [7].

2.5 Summary of the State of the Art and Identified Research Gap

The research has shown that the evolution of robot programming has evolved overtime. Early approaches use manuals teach pendant programming, followed by offline programming and recent development of automated and rule-based programming methods. Manual programming and offline programming are still widely used due to the robustness and direct physical interaction offered by programming. However, these methods require expert knowledge and therefore provide limited flexibility in industry. Offline programming optimizes some of these limitations of manual programming by enabling virtual simulation, but it remains limited by accuracy of digital model and need for manual defined task sequences. Automate rule-based program generation methods aims to reduce engineer efforts by enabling higher level system process which increases the consistency and modularity in robot programming. Nevertheless, these approaches rely on predefined rules and conservative planning assumptions, which restrict their applicability in highly variant production scenarios.

In the context of sheet metal bending and press brake automation, these limitations are prominently highlighted. The regripping, reposition workpiece for multiple bend on different sides significantly affects the reachability and grasp feasibility. Current robotic system therefore depends heavily on manually planned regripping strategies and pre-defined trajectory planning. As a result, adaptability to new part geometry remains limited, bending sequences lack flexibility and systematic reprogramming is still required when

process conditions change. Consequently, there exists a gap between existing robot programming approaches and the requirements of flexible robot sheet metal bending applications. There is a lack of methods that enable automatic program generations capable of addressing these constraints, reducing programming time, and allowing non experts operators to configure and execute bending processes. Addressing this gap is essential for increasing flexibility and adaptability in modern robotic industry

3 System Concept and Methodology

3.1 Problem Definition and Objectives

The state of the art has shown clearly that industrial robots are widely used for the press brake sheet metal bending, primarily through manual programming or predefined programming methods. These methods perform well in structured scenarios with fixed part position, geometries, and bending sequences. However, as product variants increase, the adoption of such robot programming approaches requires manual reprogramming and time consuming.

In press brake bending, after each bending operation, the shape of the sheet metal changes. Each geometric change affects robot reachability, gripping feasibility and grip point calculations, motion trajectories. Consequently, handling and regripping should be planned carefully. Previous research shows that managing this constraint is one of the most time-consuming and complex aspects of robotic bending processes particularly when handled manually.

From a methodological perspective, the limitations of this robot programming approaches related to their low-level nature. programmers must define the robot paths, position of tools and individual waypoints. This makes the programming, tightly coupled to a specific robot, grip positions or tool setups and making the reorientation or regripping strategies difficult to adapt. Robotic literature clearly indicates that such low-level programming does not scale well as system complexity increases and flexibility requirements grow [1]. This observation motivated research into the higher-level representation, where motion execution and decision making integrated with a unified framework. These methods aim to simplify handling constraints and promote reusable robot programs. While these strategies have been implemented in other robotic applications, they have not yet been implemented in robotic press brake sheet metal bending, particularly when the position and geometry changes must be explicitly considered.

Based on these observation , the central problem addressed in this thesis can be defined as follow: There is a lack of a systematic approach for automatic generation of robot programs for press brake sheet metal bending that explicitly accounts for changing workpiece positions and geometry , handling feasibility and regripping requirements while minimizing the man-

ual programming .

The primary objective of this thesis is to develop a conceptual and methodological framework for the automatic generation of the robot program that integrates bending, handling and regripping operations in one workflow. This proposed method aims to reduce manual programming effort and improve adaptability to different part geometries. To achieve this objective ,the work focuses on integrating bending sequences , handling actions , opposite side bending and regripping operation and which are validated and verified in simulation environment compatible with industrial robot environment .The scope of the thesis is limited to virtual environment and conceptual evaluation.

3.2 Requirement Definition and Management

To ensure a systematic and verifiable development of the proposed automatic robot program generation approach, the system design is guided by explicitly defined requirements. These requirements turn the research gap and problem formulation into engineering objectives that can later be verified and validated. The requirement that are explained in this chapter are directly derived from the limitations identified in the state of art and from the objectives formulated in 3.1. Rather than describing implementation details , the requirement specify what the system must achieve under which condition it must operate .This separation ensures that design decision remain traceable and that verification activities can be performed objectively. The requirement are classified into functional and non-functional requirements .Functional requirements describe the expected system behavior and capabilities , while nonfunctional requirements define the quality attributes such as usability , robustness and safety .In addition , system constraints and assumptions are explicitly stated to scope of the work and to define the operational boundaries of the proposed solution To support systematic verification, each requirement is formulated to Specific measurable Achievable Relevant Timebound (SMART) principle. Furthermore, each requirement is linked to system components and verifications steps. This traceability ensures that all requirements can be checked later and that the development process remains transparent and consistent.

3.2.1 Functional Requirements

Functional requirements specify the capabilities that the system provides in order to address the problem of automatic robot program generation for press brake sheet metal bending. These requirements define the expected system behavior from the task execution, operator interaction, trajectory generation and recovery mechanism. At a high level, the system shall enable the automatic generation of robot programs for multi-step sheet metal bending process without requiring manual t programming. The system shall support the execution of bending sequences that involve changes in workspace geometry, orientation, handling and regripping strategies.

A central functional aspects of the system is the consideration of work piece geometry evolution during bending process .Since the shape , position , orientation of the sheet metal changes after each bending operations , the system shall be feasible to handle the current geometric state and position of the workpiece when generating subsequent actions for the next task. This requirement is essential to ensure feasibility with respect to robot reachability and tool operations throughout bending sequence. Furthermore , the system shall support bending operations that requires for different sides of the work piece .Opposite side bending introduces additional constraint on tool orientation and robot configuration , which cannot always be resolved with a fixed handling strategy and time consuming .The system program therefore capable of adapting this strategy , including workpiece orientation and grip configuration, to enable such operation while maintaining feasibility

To structure the functional requirements, they are grouped into categories.

Operator Interaction Requirements: The system shall allow limited operator interaction for configuration of robot and process selection while minimizing the manual intervention during the execution. Before execution, the operator shall be able to specify the required process output, such side for bending or cad model for bending. During execution, the robot operation shall provide clear feedback or warning message when the given output conditions are unsafe for the process. The role of operator shall be restricted to configuration, while the program generation logic shall be handled automatically by the system.

Data Handling Requirements: The system should automatically use the operator input data and process the program that need to processed for

the required output in a consistent and reliable manner .This includes Handling geometries , grip position calculation, tracked poses , configuration parameters and intermediate state information generated during the time of execution. Data shall be used consistently across different stage of the program generation process to ensure that decisions regarding the handling and orientation and regripping are based on the current system state.

Motion Execution Requirements: The system shall generate feasible robot waypoints for all handling and bending related operations. This includes generation of approach waypoints, placement motions and the execution of orientation changes of the work piece and tool. Trajectory plans should be calculated prior to execution to prevent unreachable target poses. In addition, the system shall determine and apply appropriate grip positions on the workpiece to support handling and picking from different bending configurations. For opposite side bending operations, the system shall identify the grip position and waypoints such that required tool alignment can be achieved without violating the kinematical limits. Grip selection shall be treated as an integral part of the program generation process rather than manual integrated adjustments.

Error Handling and Regripping Requirements: The system shall be capable of detecting infeasible target poses or unsafe execution conditions before execution. When such conditions are detected, the system shall initiate a warning message to the user, trigger appropriate recovery actions instead of continuing execution. Recovery actions may include regripping or repositioning the workpiece to restore reachability. Regripping operation shall be supported as functional capability of the system .The system shall be able to integrate regripping sequence into generated robot program when direct trajectories to the target are unreachable or unsafe .If a critical error cannot be resolved through recovery actions , the system shall terminate execution in controlled regripping or repositioning manner to prevent unsafe robot behavior.

3.2.2 Non-functional Requirements

NFR defines the quality attributes and operation characteristics that elaborates how well the system performs its functions. While the functional requirements specify the system capabilities, non-functional requirements describe constraints related to usability, maintainability, robustness and safety.

Table 3.1: Functional Requirements for Automatic Robot Program Generation

ID	Functional Requirement (SMART)	Acceptance Criteria	Verification Method
FR-01	The system shall detect unreachable target poses before robot motion execution.	The robot shall not execute motion to unreachable poses and shall provide an error message.	Simulation based
FR-02	The system shall allow operator-side selection prior to program execution.	The selected bend side shall be applied correctly to the generated robot program.	Simulation based
FR-03	The system shall execute a regripping sequence when a target pose is unreachable within the current bend side.	Regripping shall be performed successfully and execution shall continue without interruption.	Simulation based
FR-04	The system shall handle workpiece orientation changes during program execution.	Orientation changes shall be applied correctly without violating kinematic limits.	Simulation based
FR-05	The system shall support opposite-side bending operations when required.	The system shall correctly execute opposite-side bending operations.	Simulation based
FR-06	The system shall determine and apply suitable grip positions for each workpiece state.	Grip positions shall be determined and applied correctly for each workpiece state.	Simulation based
FR-07	The system shall terminate execution when an unsafe operation is detected.	The robot shall terminate execution and display an error message.	Simulation based

These aspects are important for automated robotic systems, where corrected system behavior alone is not sufficient if the system is difficult to use, fails under complex variations or behaves unsafely during execution. The NFR of the system are derived from the operational context of press brake sheet metal bending, where industrial robot operates in close proximity to tooling, workpieces and cell boundaries. The system is intended to support flexible manufacturing scenarios with minimal operator programming while maintaining the user conditions and safe behavior. To ensure clarity, the NFRs are grouped into the following categories.

Usability Requirements: The system should be designed such that the user can operate the system easily by minimizing the operator interaction during normal operation. Configuration steps required prior to execution shall be limited to essential parameters and shall be presented in a clear and understandable manner. The system should give meaningful feedback to operator, including warning information and error messages when unreachable waypoints or unsafe condition are detected. These messages shall support the operator in understanding the cause of an issue without requiring expert knowledge of robot programming.

Maintainability Requirements: The system shall support maintainability by enabling configuration changes without requiring modifications to the source code. Process parameters, handling strategies and system setting shall be adjustable through configuration mechanisms that are separated from the core program logic. In addition, the system should provide sufficient logging or terminal output to support debugging, monitoring and future extensions. These measures are intended to facilitate adaptation of the system to new workpiece, process variations, with minimal programming effort.

Robustness Requirements: The system shall exhibit robust behavior under repeated execution cycles. Small variations in workpiece, orientation or exceptions timing or path shall not lead to system failure or unsafe behavior. The system shall handle such behavior deviation in controlled manner, ensuring smooth operations whenever feasible or triggering appropriate recovery actions when required. Robustness is particularly important in flexible manufacturing environments where idealized conditions cannot be assumed.

Safety Requirements: The system should always ensure safe operations by validating generated robot motion trajectories, informing the user about unsafe conditions and enforcing defined safety constraints and forbidden zones to prevent unreachable or unsafe poses during program execution. In the events of critical errors or unsafe conditions that cannot be resolved through recovery actions, the system shall terminate execution in a controlled manner.

3.2.3 System Constraints and Assumptions

In addition to functional and non-functional requirements, the development of the proposed automatic robot program generation system is subject to a defined sets of constraints and assumptions. These Constraints delimit the scope of the thesis and clarify the operational boundaries within which the system is designed to function. Explicitly stating these limitations ensures transparency and prevents misinterpretation of the intended system capabilities. The proposed approach is developed for a predefined robotic press brake cell configuration. The mechanical layout of the robot, press brake, tooling and auxiliary components is assumed to remain fixed throughout the system operations. Changes to physical robotic position and tool geometry, and non-collision movement planning are out of scope of this work. The system assumes the availability of accurate geometrical models of the robot,

Table 3.2: Non-Functional Requirements — NFR-01 to NFR-04 with acceptance criteria and verification methods

ID	Category	Non-Functional Requirement	Acceptance Criteria	Verification Method
NFR-01	Usability	The system shall minimise operator interaction during normal execution. Configuration steps prior to execution shall be limited to essential parameters. The system shall provide meaningful warning and error messages that do not require expert robot programming knowledge.	Operator interaction limited to bend side selection and execution confirmation. Warning messages displayed for all detected unsafe conditions.	Usability validation
NFR-02	Maintainability	The system shall allow process parameters, handling strategies, and system settings to be modified through configuration mechanisms that are fully separated from the core program logic, with no source code changes required. Sufficient terminal logging shall be provided to support systematic debugging, monitoring, and future extension of the system.	New workpiece geometry configurable via YAML file update only, without source code modification. Configuration change validated without rebuild.	Configuration validation test
NFR-03	Robustness	The system shall exhibit robust behaviour under repeated execution cycles. Small variations in workpiece orientation or timing shall not lead to system failure or unsafe behaviour. The system shall handle deviations in a controlled manner.	System completes at least five consecutive bending cycles without failure under nominal variation conditions.	Repeated run validation test
NFR-04	Robustness	The system shall tolerate moderate pose deviations within defined limits without triggering an error or aborting execution. Deviations exceeding the threshold shall trigger a warning and controlled stop.	Pose deviations within the defined tolerance threshold do not cause execution termination. Deviations exceeding the threshold trigger a warning and controlled stop.	Perturbation validation test

workpiece, press brake, and related components within the virtual environments. The correctness of reachability analysis and motion feasibility check depends on the validaty of these models. Model inaccuracies, calibration errors or unmodeled physical effects are not explicitly compensated within the proposed framework. Explicitly defining these limitations ensures transparency and prevents over interpretation of the system capabilities. The constraints are grouped into geometric, process related, cell layout, handling and safety limitations.

Sheet geometry Constraints : The system is designed for rectangular sheet metal workpiece only. All geometric reasoning, grip position calculations and edge distance check assumes a rectangular shape with four orthogonal edges. Nonrectangular geometries, polygonal shapes or free form contours are not supported. The sheet geometry is parameterized by its length, width thickness and mass. Optional circular holes may be defined using radial parameters. However, only circular holes are supported, irregular polygonal cuts are not considered. All hole's related checks are performed using radial distance evaluation. Furthermore, only a single global wall height parameter is considered during the handling. Per edge height customization is not supported.

Bend and process constraints: The bending process is restricted to 90 degrees of bend angles. Variable user defined bend angles are not supported. The bending logic assumes one bend per edge and each edge can be associated with at most one bending operation. A maximum of four bends per side is supported. For opposite side operation, the number of bends is similarly limited to four. The program logic prevents the duplicate bend assignments and multiple bend assignments on the same edge. This ensures a consistent one bend per edge configuration and simplifies the geometric reasoning required for the automatic program generation.

Robot and cell layout constraints: The robotic cell configuration is assumed to be fixed. The robot base, press brake, fixtures and auxiliary components are defined relative to a constant world coordinate frame. Relocation or reconfiguration of these elements invalidates the area logic, reachability assumptions and orientation constraints. In this case, system requires manual updates of configuration values for adaptation. Tool configuration is also assumed to be fixed. The system operates with a predefined TCP (tool center point) and attachment frame (Flange) Changes tool geometry, TCP definition or IO wiring require configuration updates and may necessitate software adaptation. Vacuum gripping is controlled via single digital output. Alternative control schemes are not considered.

Gripping constraints: The system is designed exclusively for vacuum based gripping. Grip feasibility checks are based on physical considerations including pressure difference friction and rated shear force. The holding capability of a vacuum gripper depends primarily on the achievable pressure

difference between suction cup and the ambient atmosphere, the effective contact area and the frictional interaction between workpieces surfaces and the suction elements.[10]. Mechanical magnetic or finger-based grippers are not supported within the current framework.

Safety and warning system constraints: The safety and warning logic monitors only a predefined ser of fixtures and reference objects within the cells. Full collision detection, dynamic safety trajectory planning and advanced real time safety validation are not implemented. Safety evaluation is limited to predefined geometric checks and rule-based constraints within virtual environment. Object tracking and naming assumptions -The systems rely on predefined object identifiers within the virtual environment. Specific object names are expected as key to finding to process or finding the objects. Renaming or removing these objects disrupts the workflow logic and may lead to execution failure. Therefore, renaming objects requires manual updates in the program configuration.

3.2.4 Software Platform and Tool requirement

Instead of coming from the functional analysis, the development context imposes three additional requirements. The industrial partner environment and the hardware available within the thesis scope determine the choice of execution platform, the communication interface that allows the planning system to reach the robot runtime, and the physical tool geometry from which all motion targets are computed. As a result, these limitations are formalized as SR-01, SR-02, and SR-03 software and tool platform requirements.

3.2.4.1 SR-01 Drag And Bot runtime Platform

The drag and bot software platform, developed by drag and bot GMBH. The drag and bot runtime will receive and carry out all robot motion commands, service calls, and object management operations produced by the planning modules; this work does not support any other execution environment. Two main requirements of this thesis serve as the foundation for choosing drag and bot as the execution environment. First, as mentioned in Section 3.1, the work is limited to a virtual environment, and drag and bot offers an integrated simulation runtime that enables thorough validation of robot programs without requiring access to actual robot hardware [11]. Second, the

platform allows the planning system to remain robot-independent at the software level by exposing a uniform application programming interface that abstracts manufacturer-specific robot controller syntax. By executing within the drag and bot virtual environment, the object tracking subsystem, the digital IO interface, and the object attach/detach services all operate under conditions representative of the target press brake cell, ensuring that simulation-based verification as specified for FR-01 through FR-07 is meaningful and reproducible. The suitability of runtime abstraction layers for controller-independent robot program generation is established in the modular robotic automation literature[12]. The drag and bot platform documentation confirms the availability of the virtual simulation environment, the AnyRobot Interface, and the RosBridge communication server on which the system depends. All planning modules developed are implemented exclusively against this API, and all validation test cases are executed within this environment.

Acceptance Criteria — SR-01: the planning system shall establish a stable authenticated connection to the drag and bot runtime within 5 seconds of launch; all motion, service, and IO commands shall be dispatched and acknowledged without error during the simulation

3.2.4.2 SR-02 Anybot Interface as Exclusive Communication layer.

The AnyRobot Interface, provided by drag and bot GmbH as part of the drag and bot software stack, shall constitute the sole communication channel between the planning system and the drag and bot runtime [11]. All robot motion commands: digital IO operations, object manager service calls, and operator requests shall be transmitted exclusively through this interface; direct controller-level communication or any alternative channel is outside the scope of this work. Because it combines all robot functions into a single WebSocket channel based on the RosBridge v2 protocol, such as LIN and PTP motion commands, vacuum gripper IO, kinematic object attachment, and modal operator dialogs, the AnyRobot Interface is widely used. Without a locally installed ROS environment, messages are sent as JSON-formatted objects that can be created and sent in Python. This architecture directly addresses the NFR-02 maintainability requirement by eliminating the need for a robot-manufacturer-specific post-processor and enabling the planning system to target a different robot model with just a configuration update. Connection security is enforced through a SHA-512-based rosauth authen-

tication handshake at session establishment, ensuring that only the authorized planning client can issue commands to the runtime - a design decision that supports the safety and robustness requirements NFR-03 and NFR-05.

Acceptance Criteria — SR-02: all planning-to-runtime communication shall occur exclusively over the AnyRobot WebSocket channel; an authenticated connection shall be established before any command is dispatched; LIN motion, digital IO, object manager commands shall each be successfully executed and acknowledged during the simulation

3.2.4.3 SR-03 Tool Centre point.

The tool center point (TCP) applied to all robot motion targets throughout the planning system shall be defined by the tool configuration registered in drag and bot. The TCP pose relative to the robot flange, expressed in the EULER_INTRINSIC_ZYX convention [13], is $x = 0.000\,\text{m}$, $y = 0.034\,\text{m}$, $z = 0.194\,\text{m}$, $Rz = -90.0°$, $Ry = -75.0°$, $Rx = 0.0°$. These values define the geometric offset from the robot flange frame to the contact surface center of the vacuum suction cup mounted on the end-effect. The tool center point in the world coordinate frame is where all motion targets generated by the planning system are expressed, including pickup poses, pre-bend approach poses, regrip placement poses, and post-bend unload poses. The inverse of the flange-to-TCP transform is used to derive the corresponding flange pose from the TCP target, which is then executed by the robot controller [14]. A positioning error at the workpiece contact surface results from any mismatch between the declared TCP offset and the actual gripper geometry. Such an error would jeopardize the viability of the grip and increase the risk of workpiece slippage during handling in the context of vacuum gripping, where holding force depends on the achievable pressure differential and the effective contact area [10]. By defining the TCP as a formal, traceable requirement instead of an implicit code constant, design transparency is maintained and future tool replacements are reflected as intentional requirement changes. As seen in fig 3.1 , the TCP pose values are registered in the drag and bot tool management system as project_3_tcp and are obtained from the physical geometry of the suction gripper assembly installed on the robot within the press brake cell. In accordance with the configuration-driven design required by NFR-02, these values are loaded from the input configuration file at runtime.

Acceptance Criteria - SR-03: all motion targets generated during simulation test execution shall produce flange poses consistent with the declared project_3_tcp offset; the TCP identifier and pose values shall be loaded exclusively from the YAML configuration file; a change to the TCP pose in the configuration file shall propagate correctly to all generated motion targets without requiring source code modification.

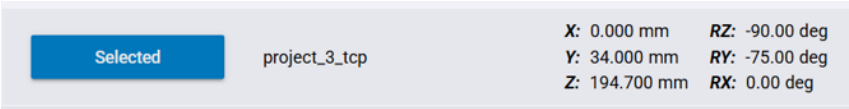


Figure 3.1: Tool Center Point parameters in Drag&Bot.

3.2.5 System Traceability

Requirements Traceability is a fundamental principle in systematic engineering development, ensuring that all defined requirements are consistently reflected in the system design, implementation and validation stages. In the context of the proposed automatic robot program generation system, traceability guarantees that the transition from conceptual objectives to executable functionality remains transparent, structured and verifiable. Each requirement is defined in section 3.2.1 (functional requirement) and Section 3.2.2(Nonfunctional requirements) is assigned to a unique identifier (e.g. FR-01, NFR -019:) .These identifiers serve as persistent reference throughout the entire development lifecycle. The identifiers are used consistently in the following aspects:

- Every requirement is implemented within a specific module.
- Every requirement is verified through a defined test scenario.
- No requirement remains unnoticed.
- No implementation functionality exists without requirement justification.

Forward traceability: confirms that every requirement is realized in a specific system components or logic module.

Backward traceability: confirms that every implemented functionality can be tracked back to an explicitly defined requirement.
By mapping each requirement to a corresponding implementation module

and validation scenario , the development process become auditable and reproducible, which is particularly important in systems involving automated decision making , such as traceability evaluation , regripping logic , and opposite side handling .In addition tractability improves maintainability and future extensibility, such as adapting to a new gripper type or extending bend angle support . This structure allows systematic impact analysis. Affected requirements and modules can be identified quickly, thereby reducing development risk.

Table 3.3: Requirement Traceability Matrix — all fifteen requirements (FR-01 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios

Req. ID	Requirement Description	Implemented In	Verified In
FR-01	Detect unreachable target poses before execution.	Reachability checking and admissibility validation logic	Simulation test case TC-01
FR-02	Allow operator-side selection of bending configuration before execution.	Operator dialog and configuration collection logic	Execution test case TC-02
FR-03	Execute regripping sequence when a target pose is unreachable.	Automatic regrip cycle logic with I/O and attach/detach control	Regrip validation test case TC-03
FR-04	Handle workpiece orientation changes during program execution.	Orientation conversion, rotation normalisation, and strategy selection logic	Orientation validation test case TC-04
FR-05	Support opposite-side bending operations when required.	Opposite-side bending workflow with sheet-flip and regrip completion logic	Opposite-side validation test case TC-05
FR-06	Determine and apply suitable grip positions for each workpiece state.	Grip planning logic with edge-clearance, spacing, and vacuum-force feasibility checks	Grip validation test case TC-06
FR-07	Terminate execution when an unsafe operation is detected.	Safety warning and pre-execution deviation checking logic with operator confirmation dialog	Safety validation test case TC-07
NFR-01	Ensure minimal operator interaction during normal execution.	Operator interaction limited to bend-side selection and execution confirmation	Usability validation
NFR-02	Allow process parameter changes without source code modification.	Configuration reader and YAML-based single-source process configuration	Configuration validation test
NFR-03	Provide clear warning and error messages for unsafe conditions.	Warning generation and modal operator notification logic	Warning message validation
NFR-04	Ensure robustness under repeated execution cycles.	Deterministic execution loop with automatic waypoint generation and path logging	Repeated run validation test
NFR-05	Tolerate moderate pose deviations within defined tolerance limits.	Configurable pose-deviation threshold and perturbation handling logic	Perturbation validation test
SR-01	drag&bot GmbH runtime platform shall be the exclusive robot execution environment.	Runtime client connection logic for communication with the configured drag&bot host and port	Platform integration check: runtime connection and command dispatch verified
SR-02	AirRobot Interface shall be the exclusive software communication layer between planning system and robot runtime.	Authenticated WebSocket communication layer with JSON command exchange	Interface integration check: authenticated connection and JSON command round-trip verified
SR-03	TCP shall be configured as project_3_tcp: $x = 0.000$ m, $y = 0.034$ m, $z = 0.194$ m, $R_x = -90.0°$, $R_y = -50.0°$, $R_z = 0.0°$.	Default TCP pose definition applied in all flange-to-TCP transformations	TCP configuration check: flange pose computed correctly for reference target poses

4 Development of Concepts

4.1 Purpose and scope of concept development

Concept development is an essential part of structured design, which consists of the design phase where different principles solutions are developed and evaluated before implementation. This is an essential part of systematic engineering design. By evaluating different options with respect to certain criteria before proceeding, the chances of selecting an inappropriate design path are minimized. REF The concept development phase is crucial for the success of the project, as it allows for the exploration of different design alternatives and the selection of the most suitable one based on a structured evaluation process.

The gap in the research, as identified in Section 2.5, reveals the absence of an effective systematic approach to automatically generate robot programs for press brake sheet metal bending. This approach should consider changes in workpiece geometry, gripping, re-gripping, and pre-execution pose, minimizing manual programming. The gap in the research is addressed in the form of measurable engineering objectives, as discussed in 3, with respect to functional requirements FR-01 to FR-07 and non-functional requirements (NFR-01 to NFR-05), which constrain how it must behave; and software platform requirements SR-01 to SR-03 (Table 3.3), which are fixed by the industrial partner environment and therefore non-negotiable. Three conceptually distinct approaches are developed in this chapter because the research gap can plausibly be addressed by different paradigms, and the selection must be justified rather than assumed

Three solution principles are proposed. The first, designated Concept A, proposes a guided teaching approach in which a structured software workflow supports the operator during conventional manual robot programming. Concept B proposes an automatic program generation system in which waypoint, grip positions , regrip sequences and opposite side handling strategies are derived automatically by evaluating the inputs configuration file , with all communication to the robot runtime conducted via the AnyRobot Interface over an authenticated WebSocket channel as required by SR-01 and SR-02.[11] Concept C is a CAD integrated offline programming in which a digital cell model is used to generate , validate and execute robot program within a simulation environment [1],[3]

The concept decision is made using Weighted Sum Model (WSM), a quantitative decision support method in which evaluation criteria are assigned relative weights and each concept is scored against criterion. This method satisfies the requirement that concept selection must be based on structure rather than on qualitative judgment. The morphological box and preferences analysis methods are alternative methods, The WSM is selected because it enables simultaneous multi-criteria weighting and produces a single quantitative score for each concept, which ensures appropriate for comparing concepts.

4.2 Concept A – Structured operator Workflow with Guided Teach in programming

4.2.1 Methodological Framework

Concept A retains the conventional teach pendant paradigm as the mechanism for defining robot motion and introduces a software-based layer that structures and sequences the operator interaction. In this method, the operator is guided through a step-by-step workflow that collects process inputs, including the desires bending sequences, bend side selection and current workpiece positions. The operator then executes each action directly via robot teach pendant or control interface using the function blocks.

This Concept is grounded in the established practice of teach pendant programming described in section2.1. As identified in the literature, teach pendant programming is reliable and widely used in structured manufacturing environment where part geometries and process parameters are fixed [1]. The guidance layer introduced in this concept (A) reduced the risk of operator errors and provides a structured interface than unguided teach pendant operation, but it does not alter the fundamental that all robot position and waypoints are defined through direct physical interaction with the robot.

Regripping operation, orientation changes, full side bending operation for a rectangular sheet under this concept A remains the responsibility of the user to define and execute. The system provides instructional prompts but does not execute, validate or generate any waypoints and position data. No interface to the robot runtime environment is required beyond standard teach pendant connectivity, so the concept does not operate within the drag and bot virtual simulation runtime and does not satisfy SR-01 or SR-02 as defined in Section 3.2.4. The system does not possess object tracking subsystems, evaluate geometric constraint, grip position calculations or assess

kinematic feasibility before execution.

4.2.2 System Structure

The system structure of concept A consists of two layers. A dialogue management layer runs on an external computer and presents the operator prompt through functional blocks or graphical interface by manual program generation. A robot controller layer receives waypoint generated manually by the operator through teach pendant and execute them. No intermediate computation layer for geometric reasoning, constraint evaluation, or motion generation exists between these layers. The architecture imposes no dependency on object tracking sensor data or trajectory generation interfaces.

4.2.3 Critical Evaluation

Concept A straightforward to implement and compatible with any industrial robot controller without software modification. The operation of the system is fully transparent and observable by the operator, which reduces the risk of unreachable and kinematics error occur during the unreachable target pose. These characteristics make it suitable for low variety, structured production scenario where process parameters are stable across batches.

However , the fundamentals limitation if this concept are significant in the context of research gap ,programming effort is not yet reduced because knowledge of robot kinematics , tooling constraints , regripping strategies and bending sequence algorithm should need to applied by the operator for every session [1], [3]. Minor changes in sheet geometry , workpiece holder placement requires full re teaching of the affected trajectories , which contradicts the flexible objective stated in section 3.1 [1], REF 26. Regripping operation and full side bending operations must ne manually planned , which introduces a chances of operator error and increase the cycle time .No pre execution validation of target poses is performed , kinematic configuration are discovered only during the physical or virtual robot motion which leads to safety risk and a source of unplanned downtime [1].

The software guidance layer of concept A provides process management but no automation level, it addresses the unstructured operator workflow without addressing root cause, which is the requirement for manual position definition. Concept A does not contribute to the primary objective of automatic program generation and failed to satisfy most of the functional and non-function requirements defined in 3.1. and REF NFR.

4.3 Concept B – Automatic Program Generation based on rules with Integrated Constraint Validation.

4.3.1 Methodological Framework

Unlike Concept A, which retains manual position definition, Concept B eliminates it entirely. Concept B proposes an automation level, rule based, automated program for generation system in which all robot waypoints, grip positions, regrip sequences, opposite side handling operations and strategies are derives automatically from geometric input parameters and a structures YAML configuration file: operator interaction is limited selection of required output selection before execution and confirmation of generated programs visually. No manual waypoint generation, direct teaching or any runtime operation or decisions are required during program execution.

The system communicates with the robot runtime through AnyRobot interface, a robotics application program (API) Developed by KEBA \ Drag and Bot GmbH that provides standardized method for controlling industrial robots from different manufacturers via WebSocket communication[11]. The planning Software operates as the WebSocket client, the drag and bot runtime act as the server. This architecture establishes a clean separation between the planning computation layer and robot execution layer, which is a recognized design principle [12] .

Workpiece pose data is retrieved in real time from drag and bot object tracking subsystem and processes by an orientation validation component , which determines whether the tracked sheet orientation is admissible for the planned bending operation by applying a three level priority strategy ,a side normalized orientation is evaluation first , followed by original tracked orientations by the object tracker and finally a 180 degree rotated strategy , when neither of the first two falls within the configurable window. This Three level hierarchy ensures that valid gripping orientations and positions are identified for any initial sheet placement on the cell floor, which eliminates the need for operator prepositioning. Before grip position calculation, the program validated the sheet position is within the robot cell. This is applied to avoid the kinematic reachability error occurred during the program execution. The zone-specific admissibility limits are specific limits are robot cells configuration are configured in the input sections and require no code modification when cell layout changes. This design decision satisfies NFR 02as defined in table 3.2 and NFR 05 as defined in table REF TABLE NFR .

Grip position are computed by a parameterized grip planning component s that uses the sheet dimension, mass suction cup diameter , rated shear

force and edge clearance margin which all are defined in the input configuration files :candidate grip position reevaluated against edge clearance and cup spacing constraints and against the vacuum holding force which is derived from the relationship between suction pressure , cup contact areas and dynamic loading during transport [10].Grip feasibility is evaluated before kinematic reachability assessment, ensuring that only variable grip configurations are submitted to the motion generation layer. This ordering prevents unsafe handling states that would result from a grip position that is kinematically reachable but physically unable to reach the sheet during transport [10] .

When the current grip configuration does not allow the targeted pose to be reached, an automated regrip cycle is initiated. The cycle places the sheet on a designated orientation table, repositions the robot to reset and then regrasp the sheet at an orientation that restores full reachability. This regrips logic directly satisfies FR03 as defined in table 3.1 .

Opposite side bending is also supported through this concept through a dedicated module that executes a structured sheet flip sequence. After all primary bending operations are completed, the program evaluates whether the operated has selected for opposite side bends. If so, the sheet passes to flip position through a coordinated sequence of motion, gripper activation, and object attachment operations targeting positions to flip the sheet. Afte the flip, the opposite bending sequence is executed under the same validation, including orientation normalization, grip feasibility evaluation and reachability assessment before the waypoints is generated. This enforcement of validation across both primary and opposite side bending operation ensures consistent safety behavior throughout th full step bending process [7]

The motion trajectories are generated in linear cartesian motion commands using minimum jerk velocity interpolation profile REF 16: This profile minimizes the third derivatives of position over the motion, producing smooth velocity transition between waypoint that reduce peak acceleration and gripper loading forces during sheet transport. This is requirement for maintaining vacuum grip stability under dynamic conditions [10].A singularity avoidance strategy ensures the robot arm does not pass through kinematically degenerate configurations during the approach phase [1]. All motion targets are expressed relative to the pre-defined tool center point registered in the drag and bot tool management system. This reference is mandatory because SR-03 requires that all generated flange poses be consistent with the physical gripper geometry of the KEBA press brake cell, and the defined

TCP entry encodes that geometry with the exact pose values specified in Section 3.2.4 [11].

4.3.2 system structure

The structure of Concept B consists of Five layers as defined in section System-Architecture-and-Design. The input and configuration layer that collects all required inputs for the programming. The geometry and constraints evaluation layer applies grip feasibility rules, edge clearance checks and zone-based orientation constraints. The planning and decision logic layer determines the execution strategy, which decides the program to be direct execution or regrip required execution or workpiece flipping strategy, based on reachability and orientation validity . The trajectory planning layer produces linear cartesian waypoints with minimum jerk velocity profiles. The safety and validation layer applies pre-execution pose check before the command is published to the robot runtime.

This WebSocket channel of the Any robot interface allows to connect all five layer to drag and bot virtual simulation run time [11] .The configuration-driven approach allows all process parameters to be trimmed from program logic, allowing the solution to tackle new workpiece geometries simply by modifying the content of configuration files. This was in direct satisfaction of non-functional requirement NFR-02 as described in Section 3.2.2.

4.3.3 Critical evaluation

Concept B is aligned with the research gap stated in Section 2.5 for every dimension identified in the literature. Define waypoints, plan regrips, and sequence work on the opposite side automatically from geometric parameters instead. A pre-execution pose validator guards against unsafe execution states before motion starts. This allows for configuration driven architecture that can adapt to new part geometries without the need for source code changessatisfying FR-01, NFR-03, NFR-04, and NFR-05. Configuration-driven parameterization satisfies NFR-02. The system operates exclusively within the drag and bot runtime via the AnyRobot Interface, satisfying SR-01 and SR-02. All motion targets reference the pre-defined tool center point, satisfying SR-03. All fifteen requirements in Chapter 3 are therefore addressed by the concept design.

The primary constraints that Concept B operates within are those identified within Section 3.2.3. The system operates within a rectangular sheet geometry with fixed 90-degree bend angle constraints. The integration with

the KEBA robot controller hardware, which is part of the KEBA robot driver layer within the drag and bot software stack [11], is identified within the future work section and is not included within the scope of this thesis. All validation has been conducted within the drag and bot virtual simulation environment, which is discussed in this section. These constraints are explicit and not indicative of design flaws but, rather, define the scope within which the design concept operates.

4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation

4.4.1 Methodological Framework

Concept C proposes a computer aided design (CAD) integrated offline programming (OLP) approach in which a digital model of the sheet metal workpieces and the robot cell used to plan , generate and validate robot program in a simulation environment , without using physical robot interaction during the programming .The OLP software evaluates bending sequences , grip positions and regrip programming, kinematic reachability and genreted robot program based on the CAD geometry of the workpiece and tooling .The generated programs are validated inside a simulation before transfer to the robot controller for execution.

This approach corresponds to the offline programming section 2.2. Industrial Olp tools such as ABB Robot Studio ,KUKA: sim And FANUC ROBOGUIDE implement this approach and are used in industry for robot program generation in structured production environment [3] .The digital twin based evaluation and validation reduces the occurrence of risk by identifying unreachable poses and collision condition before real execution, which has been identified as significant advantage over the teach pendant programming. [15]

4.4.2 System Structure

This concept consist of four layers .A cad modelling layer provides digital representations of the robot , press brake , tools and workpiece .An OLP and simulation layer uses the URDF(Unified Robot Description Format) representations to plan the algorithm for robot program through built in kinematic solver .A post processing layer converts the generated programs into controller specific syntax’s robot controller layer communicates the program for exception. Accurate digital twin representation of all cell components are required at the modelling layer small errors in modelling the digital twin

model propagate as execution error that require manual re programming during commissioning.[1] [3].

4.4.3 Critical Evaluation

Concept C offers meaningful automation advantages for structured, high-volume production environments where accurate cell models are available and OLP tool investment is justified. Simulation-based validation provides strong pre-execution assurance, and the approach is supported by mature commercial software [1][15]. These characteristics make it the dominant paradigm in industrial robotic bending automation at the time of writing.

For the scope of this thesis, Concept C presents three fundamental disadvantages that result in its rejection. First, the implementation requires accurate CAD models of the complete robot cells, including the robot, press brake, tooling, fixtures, and workpieces — which represents a substantial engineering preparation effort that is not available within the thesis timeframe.

Second, commercial OLP systems involve high license fees while demanding expert-level instruction - resources simply unavailable within this project's constraints REF Engineering Design: A Systematic Approach . Besides cost, mainstream OLP platforms lack native compatibility with both the drag and bot simulation framework and the AnyRobot WebSocket system, which define the core setup targeted here [11]. Connecting such a tool would mean building a bespoke bridge to the drag and bot engine - an extra burden unjustified by timeline or ambition.. Even if the first two limitations were resolved, this third limitation would disqualify Concept C, because SR-01 and SR-02 are non-negotiable constraints from the industrial partner environment.

4.5 Structural Comparison of Concepts

Before applying the quantitative WSM, the three concepts are compared at the structural level to make the qualitative differences explicit and to verify that the comparison covers all requirement categories. This structural comparison explicitly includes SR-01 and SR-02, because platform compatibility is a binary disqualifier rather than a continuous criterion - a concept that cannot operate within the required platform fails the evaluation regardless of its scores on other dimensions. Table 4.1 presents this comparison across nine features. Table 4.1 makes the selection case for Concept B unambigu-

Table 4.1: Structural comparison of Concept A, Concept B, and Concept C across key design features

Design Feature	Concept A	Concept B	Concept C
Waypoint Generation Method	Manual operator teach-in for every new part geometry [1]	Automatic derivation from sheet geometry and YAML configuration parameters [11]	CAD-based OLP solver algorithms; new CAD model required per variant [3]
Regrip Automation	No automation. Operator plans and executes each regrip sequence manually [2]	Full automation via dedicated regrip module: IO control, attach/detach, and rotation sequencing under YAML-configurable parameters [2][7]	Regrip positions defined in CAD simulation; execution not automated at runtime [15]
Opposite-Side Bending	Manual sheet flip by operator; no software support	Dedicated flip module executes structured sheet transfer via predefined cell objects; same validation hierarchy as primary bending [7]	CAD simulation resolves flip; runtime execution remains manual [15]
Pre-Execution Pose Validation	None. Unreachable poses discovered at runtime during physical robot motion [1]	Orientation validation evaluates admissibility using a three-level priority strategy	Orientation validation performed in CAD simulation; no runtime checks [3]
Configuration Adaptability	Not supported. Full re-teaching required per geometry variant [3]	sheet dimensions, bend sides, and grip parameters configurable without code modification	CAD-based approach requires new model and re-simulation for each variant [15]
Platform Compatibility	No dependency on drag and bot or AnyRobot Interface; operates via teach pendant only. Does not satisfy SR-01 or SR-02	ROS-based interface with drag&bot AnyRobot middleware for real-time command execution [11]	Incompatible: standard OLP tools do not support direct integration with the drag and bot runtime or AnyRobot Interface; violates SR-01 and SR-02
Tool Centre Point Definition	TCP not software-defined; operator sets tool geometry manually on teach pendant. SR-03 (project_3_tcp) cannot be enforced	All motion targets computed relative to the project_3_tcp TCP registered in the drag and bot tool management system, satisfying SR-03	TCP defined within OLP CAD model; independent of the project_3_tcp constraint required by SR-03
Implementation Scope Feasibility	High feasibility but provides no automation contribution to thesis objective	Fully implementable in Python within dragand bot virtual simulation; achievable within thesis scope [11]	Requires commercial OLP software, accurate cell CAD models, and custom integration; exceeds thesis scope [1]
Research Gap Addressed	No automation; manual processes dominate	Automated waypoint generation, regrip sequencing, and flip execution with real-time validation; addresses key automation gaps in current state of the art [7]	Partial - Focuses on CAD-based simulation and OLP without runtime automation; does not address real-time adaptability

ous at the structural level. Concept A achieves a positive rating only on implementation feasibility, because its teach-pendant architecture requires no additional tooling - but this advantage is outweighed by its failure on all five automation-related features and both platform requirements SR-01 and SR-02. Concept C achieves partial positives on waypoint generation and pre-execution validation through its CAD-based approach, but these are negated by the SR-01 and SR-02 incompatibility, which is a structural disqualifier rather than a weighted disadvantage. These structural observations confirm in advance the numerical outcome of the WSM presented in Section 4.6

4.6 Concept Decision-Weighed Sum Model (WSM)

4.6.1 Methodology

The quantitative concept selection technique used is the Weighted Sum Model. Each evaluation criterion in the WSM is given a weight based on how important it is to the design objectives. Each concept is then scored against each criterion on a scale of 1 to 5, with 5 denoting full satisfaction and 1 denoting non-fulfillment. Each concept's overall weighted score is calculated by multiplying the corresponding score by each criterion weight. The idea that receives the highest total weighted score is chosen.

4.6.2 Evaluation Criteria and Weights

Six evaluation criteria are defined directly from the functional, non-functional, and software platform requirements of Chapter 3. The weight assigned to each criterion reflects its relative contribution to resolving the research gap identified in Section 2.5, as justified below

Programming Effort Reduction (25%) -This criterion carries the highest weightage because it directly addresses the primary research gap. This makes automation easier by reducing a lot of manual programming especially when parts change frequently. This gap is consistently identified in literature as the most significant barrier to increased automation in robotic bending cells.

Automated Regrip and Opposite-Side or sheet flipping Support (20%) : According to Section 2.4, the most difficult and time-consuming parts of robotic press brake bending that are not sufficiently handled by current methods are regripping and opposite-side handling[2][7]. The 20% weight reflects that automating these operations is a core deliverable, not a secondary feature, of the research gap resolution per FR-03 and FR-05 in Table 3.1.

Pre-Execution Pose Validation (20%) : Before robot motion is started, infeasible poses must be identified in accordance with safety regulations NFR-03, NFR-04, and NFR-05. In robotic bending cells, executing unreachable target poses is a major source of system failure as well as a safety risk.

Flexibility for Changing Part Geometries (15%) : One of the main goals derived from the research gap is the capacity to adjust to new workpiece variations without reprogramming. NFR-02 is directly satisfied by configuration-driven adaptability, and the 15% weight indicates that this is a secondary goal following the primary automation requirements.

Platform Compatibility drag and bot Runtime and AnyRobot Interface — SR-01, SR-02 (10%) :The industrial partner environment mandates the drag and bot virtual simulation runtime and the AnyRobot Interface as the exclusive execution and communication platform, formalised as SR-01 and SR-02 in Section 3.2.4. A concept that fails SR-01 or SR-02 cannot be implemented or validated within the thesis scope regardless of its theoretical merits, which is why a score of 1 on this criterion effectively disqualifies a concept. SR-03 (project_3_tcp) applies equally to all platform-compatible concepts and therefore does not independently differentiate them; it is reflected in the Concept B system structure in Section 4.3.2.

Operator Usability (10%) : Non-functional requirement NFR-01 says that the system must be usable by non-expert operators with little interaction during normal operation. The 10% weight shows that usability is an important quality attribute, but it is not as important as the main automation requirements.

4.7 Result of the weighted Sum Model

Table 4.2 presents the complete WSM evaluation with score justifications and weighted totals for all three concepts

4.8 Analysis of results and Concept Decision

The WSM produces total weighted scores of 1.40 for Concept A, 5.00 for Concept B, and 2.70 for Concept C. Concept B achieves the maximum possible score and is therefore selected as the concept to be developed and implemented in Chapter 5. The margin between Concept B and the nearest alternative is 2.70 points, which is not a marginal result: it reflects systematic superiority across every criterion, not a single dominant factor.

Concept B meets all six requirements to the highest degree, as indicated by its score of 5.00. Because waypoints are automatically derived from geometric parameters and there is no manual teaching step, programming effort

Table 4.2: Structural comparison of Concept A, Concept B, and Concept C across key design features

Evaluation Criterion	Wt.	Concept A — Justification	Sc.	Concept B — Justification	Sc.	Concept C — Justification	Sc.
Programming Effort Reduction	25%	All waypoints defined manually per session; no reduction in programming effort	1	Waypoints derived automatically from geometric parameters; no manual teaching required	5	CAD solver reduces teaching effort but requires model setup per variant	3
Automated Regrip and Opposite-Side Support	20%	No regrip logic; operator plans and executes all regrip and flip operations manually	1	Full automation via regrips and opposite-side modules with IO and attach control	5	Regrip targets defined in simulation; execution not automated at runtime	2
Pre-Execution Pose Validation	20%	No validation; unreachable poses detected only during physical robot motion	1	Orientation validation and spatial zone classification executed before motion generation	5	Simulation validates program before export; no runtime check available	3
Configuration Adaptability	15%	Full re-teaching required per geometry variant; scales poorly	1	Sheet parameters stored in YAML; new geometry requires only configuration update	5	New CAD model and re-simulation required for each variant	3
Platform Compatibility	10%	No dependency on drag&bot or AnyRobot Interface; incompatible with the required industrial partner development platform. SR-01 and SR-02 not satisfied	4	Fully implementable in Python within drag&bot virtual simulation; achievable within thesis scope	5	Standard OLP tools cannot integrate with drag&bot runtime or AnyRobot WebSocket Interface; violates SR-01 and SR-02	2
Operator Usability	10%	Expert robot programming knowledge required for each session; high cognitive load	2	Operator selects bend side and confirms execution; decisions handled by the system	5	CAD interface reduces robot expertise requirement; specialist CAD knowledge required	3
Weighted Total Score	**100%**	Concept A	**1.40**	Concept B — SELECTED ✓	**5.00**	Concept C	**2.70**

is eliminated. Because specialized software modules implement gripper IO control, object attach and detach sequencing, and multi-step rotation under YAML-configurable parameters with all behavior fully specified before run-time, regrip and opposite-side bending achieve full automation. Orientation validation and spatial zone classification are two independent components that structurally guarantee pre-execution validation because they are both called before any motion command is generated, preventing an invalid pose from reaching the motion generator. NFR-02 is satisfied by configuration-driven sheet parameterization. As required by the industrial partner, the system satisfies SR-01 and SR-02 by operating solely within the drag and bot runtime via the AnyRobot Interface. SR-03 is satisfied since all motion targets make reference to the project_3_tcp tool center point. Within the parameters of the thesis, the Python-based implementation is completely feasible [16]. By restricting interaction to necessary pre-execution choices,

the dialog-based operator interface complies with NFR-01.

Because Concept A does not meet the five most important requirements, it receives a score of 1.10. Regardless of any other characteristics, its platform compatibility score of 1 indicates that it is incompatible with the industrial partner development platform since it does not function within the drag and bot runtime, meeting neither SR-01 nor SR-02. The concepts teach-pendant architecture prevents it from contributing to any aspect of the research objective, as evidenced by the scores of 1 on programming effort, regrip automation, pose validation, and configuration adaptability [16].

Because Concept C's CAD-based OLP approach tackles waypoint generation, pre-execution validation, and configuration adaptability during the simulation phase, it receives a score of 2.70. However, the CAD approach's lack of runtime constraint evaluation and requirement for per-variant model rebuild rather than configuration file update limit the score. The platform compatibility score of 1 indicates a structural disqualifier: standard OLP tools are unable to integrate with the AnyRobot WebSocket Interface or the drag and bot runtime, which violates SR-01 and SR-02. This platform incompatibility alone would prevent Concept C from being implemented or validated within the thesis environment, even putting aside the resource and cost constraints [1] [16].

Three conceptually different methods for automatic robot program generation in press brake sheet metal bending were developed and assessed in this chapter. The evaluation adhered systematic engineering design methodology because it offers an organized framework for evaluating options based on predetermined criteria prior to committing to implementation, which is necessary for a thesis-level concept decision [16]. All three of Chapter 3's requirement categories—functional requirements FR-01 to FR-07 (Table3.1), non-functional requirements NFR-01 to NFR-05 (Table 3.2), and software platform requirements SR-01 to SR-03 (Table 3.3, Section 3.2.4)—were used to develop the evaluation criteria.

Because Concept A operates independently of the drag and bot platform and AnyRobot Interface required by the industrial partner environment, its teach-pendant architecture cannot reduce programming effort, provide automated constraint validation or regrip support, or satisfy SR-01 or SR-02. As a result, the guided teach-in approach was rejected. These failures are structural: according to Table 3.1, FR-01, FR-03, FR-05, and FR-06 require automated geometric reasoning, which the two-layer teach-pendant architecture is unable to supply.

Concept C,method of programming offline: Integration with CAD soft-

ware for creating digital cell models for use in offline programming (OLP). This method was disregarded because required commercial OLP tools do not exist; accurate digital cells are also not included in the thesis scope. Therefore, the approach as a practical limitation has a structural constraint due to its incompatibility with drag and bot's run-time and the AnyRobot Interface (Section 3.2.4, SR-01, SR-02) [1][3]. Consequently, any partial automation benefits associated with this method could not be achieved using simulation techniques due to these constraints also being exhibited in the simulation environment within this thesis.

The rule-based automatic program generation system of Concept B was selected because it achieves the maximum weighted score of 5.00 in the WSM, satisfying all fifteen requirements of Chapter 3. This automated system eliminates programming time through automatic determination of waypoints, automates regrip and opposite side bend sequences through dedicated software modules, and verifies whether there is an admissible orientation and spatial reachability prior to the generation of any motion; (b) operates solely in the drag and bot virtual simulation runtime, through use of the authenticated AnyRobot WebSocket Interface. This satisfies both SR-01 and SR-02. All motion targets are computed with respect to the project_3_tcp (tool centre point); therefore, it meets requirement SR-03. The technical details about Concept B, including the entire software architecture, interface specifications, algorithm derivation and risk analysis can be found in Chapter 5.

5 Implementation

5.1 Introduction and scope

The development of product phase transforms the selected concept into a fully realized technical system. Concept B, chosen through the weighted sum model with a total weighted score of 5.00. Exceeding Concept A at 1.60 and concept C at 2.70 is implemented here as an executable software architecture that generate robot programs automatically for press brake sheet bending. Concept B was the only concept which satisfies most of the requirements as defined in chapter [3]. The implementation addresses four accountability domains. The first is the system architecture, covering the layered module structure and rationale for every structural decision. The second is the interface specification, describing all data exchanges between modules and between planning software and drag and bot runtime. The third is the design failure mode and effect analysis in section, which links each failure mode to a risk priority number and countermeasure, fulfilling the peruse risk assessment obligation. The fourth is the software implementation record, presenting algorithmic derivation and configuration structures so that every design is traceable to a requirement. The system operates within the drag and bot virtual simulation environmental communication with the robot runtime is conducted through the Any Robot Interface over a persistent authenticated WebSocket Channel, satisfying all the software requirements' process parameters are loaded from a single input configuration file at the staring , satisfying NFR-02 by enabling workpiece changeover through a file edit alone .Operator input is provided through a purpose built interactive 3D sheet bending viewer embedded in the drag and bot operator cockpit , which give a visual geometry feedback reduces the risk of incorrect bend side selection . Ubuntu Linux is required as the development platform because drag and bot is built on ROS, which has no production release for Windows or macOS [17] Fig 5.1 shows the complete bending cycle execution from session start through workpiece gripping. Each stage maps directly to a section of this chapter. The chapter is organized as follows, system architecture, module overview, communication stack configuration architecture and operator interface are presented first. The simulation Environment and spatial zone classification flow. Each functional subsystem is then developed in the execution of a bending session, covering pre-execution safety valida-

tion, workpiece orientation strategy, gripper control, grip position planning, regrip sequencing, sheet flipping and bending, trajectory generation and post bend unloading. A design Failure mode and effect analysis concludes the implementation record and the final section maps all the requirements from chapter 3 to their implemented subsystems.

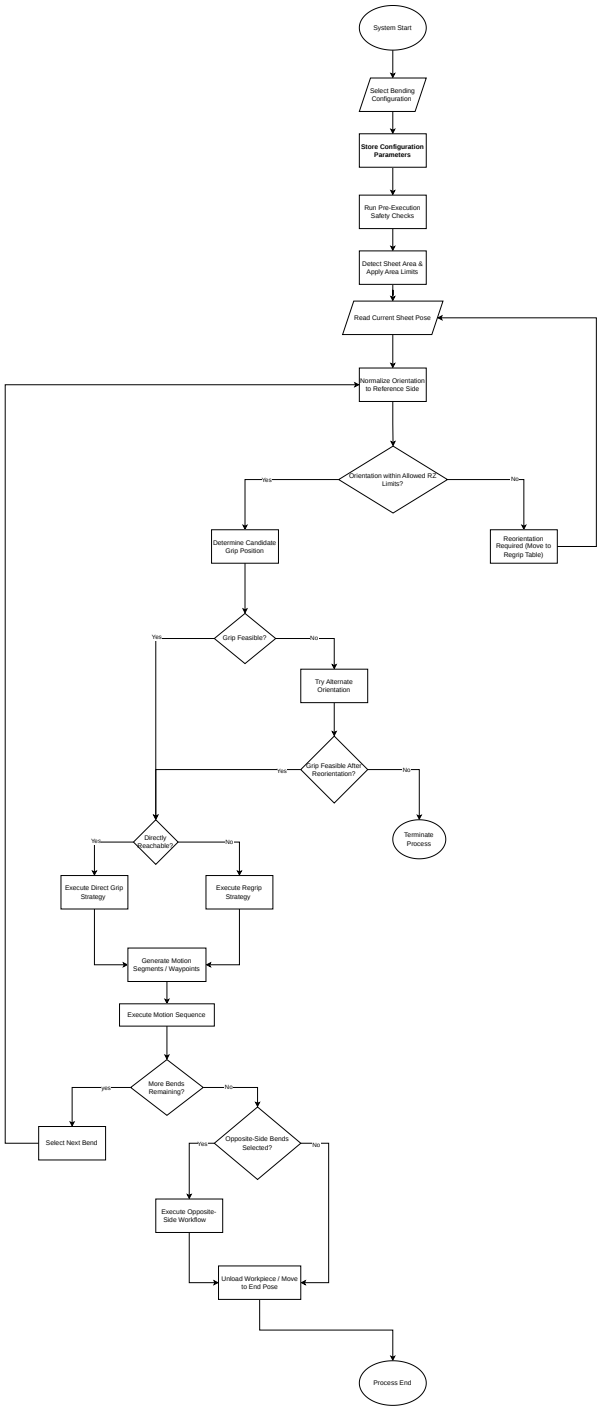


Figure 5.1: System architecture

5.2 System architecture

5.2.1 Layered Architecture Design and data flow

The system adopts a layered architecture in which each module depends exclusively on module in the layered directly beneath it. This pattern is selected because the press brake automation context imposes three constraints that unstructured flat module design cannot satisfy simultaneals, independently testable constraints evaluation, maintenance safe replicability and structural enforced safety ordering.

Testability is achieved by enabling the module to be evaluated using synthetic pose data and YAML-based configuration inputs, without the need for an active WebSocket connection. This isolation is essential for verifying the orientational classification mathematics and grip feasibility calculations, because that computation cannot be inferred correctly from a live system alone. Replaceability is ensured by a modular architectural approach in which changes to the grip position algorithm are localized within the constraint evaluation layer. The planning and motion layers are structurally isolated and maintain no direct dependencies on constraint evaluation modules, thereby satisfying the requirements of NFR-02. Safety ordering is maintained through a structured design in which pre-execution safety validation is implemented in an independent layer executed prior to any motion commands. This approach ensures that future modifications cannot inadvertently reposition safety checks after motion initiation, thereby satisfying FR-07 and NFR-03.

Five distinct layers are defined within the system architecture, each with a clearly assigned responsibility. Layer 1 (Input and Configuration) is responsible for acquiring all required parameters from the input file, along with the operator's bend-side selection, prior to the establishment of any WebSocket connection. Layer 2 (Geometry and Constraint Evaluation) performs the computation of geometric and physical constraints based on live tracker data, while remaining strictly limited to evaluation tasks and producing no motion commands. Layer 3 (Planning and Decision Logic) determines the overall execution strategy and coordinates the interaction between Layers 2 and 4, without directly generating motion instructions. Layer 4 (Motion Generation and Communication) serves as the exclusive interface for robot communication, generating LIN and PTP waypoints, converting them into JSON format, and transmitting them via an authenticated WebSocket connection. Layer 5 (Safety and Validation) is invoked at the beginning of every session and enforces a mandatory validation step, preventing further execution

until explicit operator confirmation is obtained. The complete architectural structure is illustrated in Figure

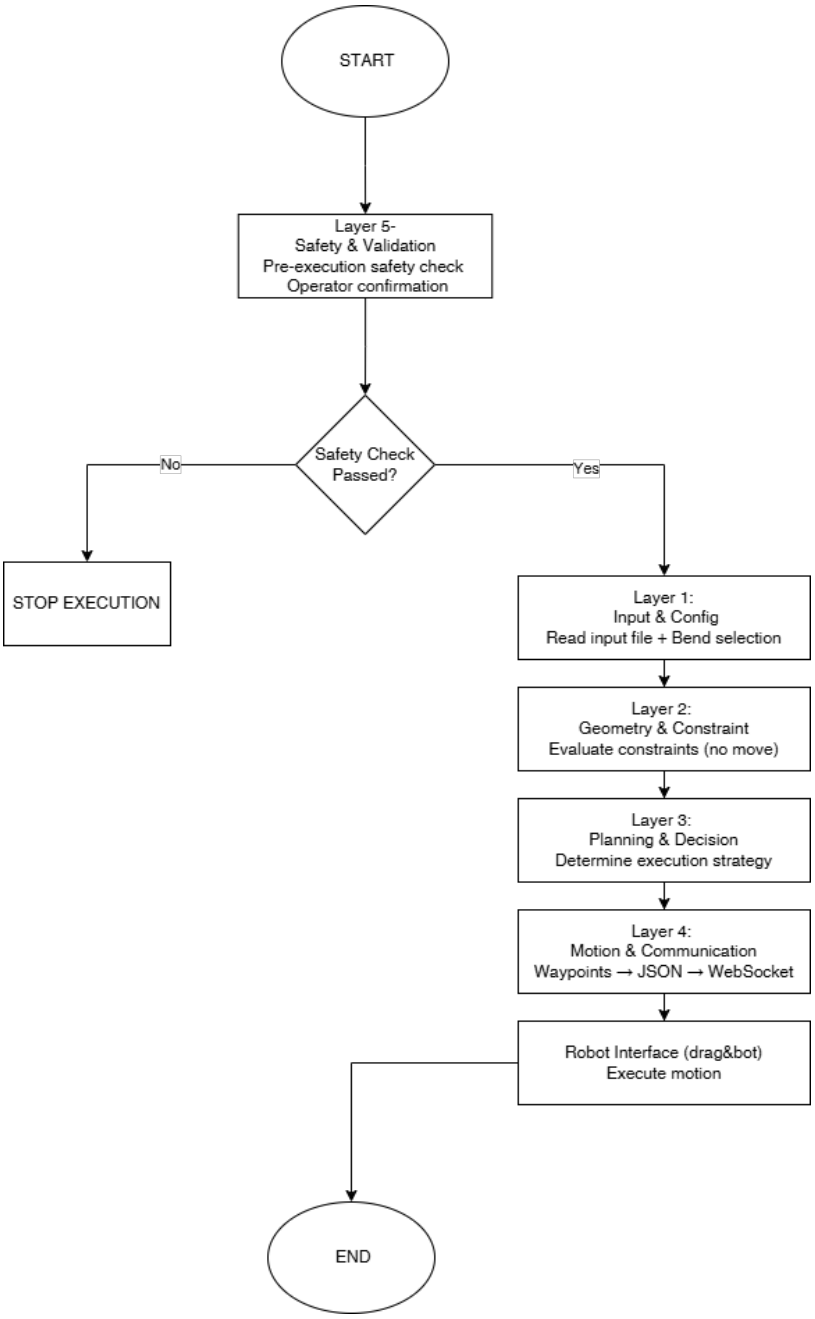


Figure 5.2: Five layer architecture

The progression of data throughout a bending cycle follows a structured transformation across the system layers. At the initial stage, sheet geometry, bend-side selection, and TCP parameters are provided as input. These inputs are subsequently interpreted through geometric and constraint-based evaluation, where raw tracker pose data is converted into a validated pickup orientation and a feasible grip position. Based on this processed information, an appropriate execution strategy is established before being translated into

motion commands, which are encoded in LIN-based JSON format and transmitted to the runtime environment.

The reliability of the overall process depends on a small number of critical transformation stages. Any inaccuracy introduced at these points propagates forward and affects all subsequent computations and decisions. In particular, the correct determination of pickup orientation is essential for successful workpiece acquisition. Likewise, the evaluation of grip feasibility ensures stable handling under dynamic conditions and prevents slippage during motion. During trajectory generation, a balance must be maintained between path fidelity and the limited processing capacity of the robot controller. This requires a controlled reduction of trajectory points while preserving the essential characteristics of the intended motion.

5.2.2 Module Overview and dependency Discipline

The implementation consists of software modules organized across the five system layers. Table 5.1 summarizes each module in terms of its layer assignment, functional role, and primary responsibility. Each module is defined with a single primary responsibility to ensure clear fault attribution during failure analysis. Assigning multiple responsibilities to a single module would make it difficult to associate a specific failure mode with a well-defined functional unit.

Table 5.1: Software modules across system layers and their primary responsibilities

No.	Layer	Module	Primary Responsibility
1	L1	Session launcher	Coordinates YAML loading, 3D viewer initialisation, and session start-up before runtime connection.
2	L1	Configuration reader	Loads and validates configuration data and provides a consistent parameter source for the remaining modules.
3	L5	Safety checker	Enforces operator confirmation and safety validation before execution begins.
4	L2	Zone classifier	Classifies tracker data into spatial zones and determines admissible orientation strategies.
5	L2	Orientation selector	Determines the pickup orientation on the basis of geometric evaluation of the workpiece.

No.	Layer	Module	Primary Responsibility
6	L2	Face orientation converter	Converts tracker pose data into a reference-face representation using rotation transformations.
7	L2	Grip position planner	Computes feasible grip positions while considering edge clearance and holding constraints.
8	L2	Pickup orientation validator	Verifies tool alignment and applies corrective transformations where required.
9	L4	Trajectory generator	Generates waypoint sequences with path refinement, sampling control, and singularity avoidance.
10	L4	Regrip motion executor	Executes regrip operations, including transport, release, and re-acquisition.
11	L3	Primary bending orchestrator	Coordinates execution flow across orientation, planning, and motion modules.
12	L3	Regrip sequencer	Determines transition sequences between grip configurations.
13	L3	Opposite-side runner	Evaluates and executes alternative orientation strategies when required.
14	L3	Multi-stage regrip loop	Handles iterative repositioning for complex bending sequences.
15	L3	Post-bend unload sequencer	Controls unloading of completed workpieces to the designated buffer location.
16	L4	Waypoint data structure	Encapsulates robot motion parameters, including pose, velocity, and blending information.
17	L4	Digital IO handler	Manages vacuum control and safety-related input/output signals.
18	L4	WebSocket client	Handles authentication, session management, and message transmission to the runtime interface.
19	L4	Command type definitions	Defines motion categories, coordinate conventions, and JSON message formats.

Three structural properties ensure that the architectural constraints are consistently enforced across the system.First, the WebSocket client and the command type definitions together constitute the exclusive communication layer. No mod-

ule outside Layer-4 constructs WebSocket messages or performs direct network operations. As a result, authentication handling and connection recovery are implemented centrally and remain consistent across all runtime interactions. Second, the safety checker is positioned within Layer 5, which is not imported by any lower layer. This separation prevents planning or execution modules from invoking safety validation during runtime. Instead, safety checks are enforced strictly at the beginning of a session, ensuring that they cannot be bypassed or reordered during execution. Third, the trajectory generator serves as the sole interface for dispatching motion commands to the runtime. All waypoint sequences are routed through this module, ensuring that workspace clamping and singularity checks are consistently applied before execution. This guarantees that all generated motions comply with the defined safety and feasibility constraints.

5.2.3 Communication Stack, Authentication interface Specification

All runtime communication passes through the AnyRobot Interface, selected because it is the only supported external API for drag and bot. Direct ROS client libraries are excluded because they would couple the planning software to a specific ROS version, violating NFR-02 [11] [17]. The AnyRobot Interface translates JSON over a persistent WebSocket channel into native ROS operations using the RosBridge v2 protocol, enabling the entire planning software to run in standard Python 3 without a ROS installation on the planning machine.

WebSocket (RFC 6455) is chosen over HTTP because it supports a persistent connection that allows continuous data transfer. In contrast, HTTP requires a new TCP connection for each request, which introduces a delay of approximately 3–15 ms per batch. When sending a trajectory of 400 points in batches of 50 commands, these delays accumulate to 24–120 ms. This added delay can cause the robot controller's command buffer to run empty, leading to interruptions during motion execution. By maintaining a continuous connection, WebSocket avoids this overhead and ensures smooth and uninterrupted command transmission.

Figure 5.3 presents an overview of the system communication architecture. The planning software establishes a persistent WebSocket connection to the robot controller via a defined network endpoint (e.g., `ws://192.168.x.x:9091`). Communication is organized through a set of dedicated interfaces, each responsible for a specific function within the system. The Ros Bridge v2 interface is used to transmit motion commands, while the `rosauth` service ensures secure authentication using the SHA-512 hashing algorithm. This method encodes authentication data into a non-reversible format, preventing the transmission of sensitive information in plain text. Motion trajectories are transmitted in batches via the `/command_list` interface, and real-time object information is retrieved through `/dnb_object_manager/list_objects`. Gripper control is performed using `/robot/set_digital_ios`, whereas object attachment and detachment are handled

through the corresponding manager interfaces. User interactions, such as confirmations and prompts, are managed via the `human_interaction/request_hi` interface. The YAML configuration is not part of the runtime communication. It is loaded once during system initialization and shared across the relevant modules. This approach ensures that missing or incorrect parameters are detected early, before any communication with the robot is initiated [18].

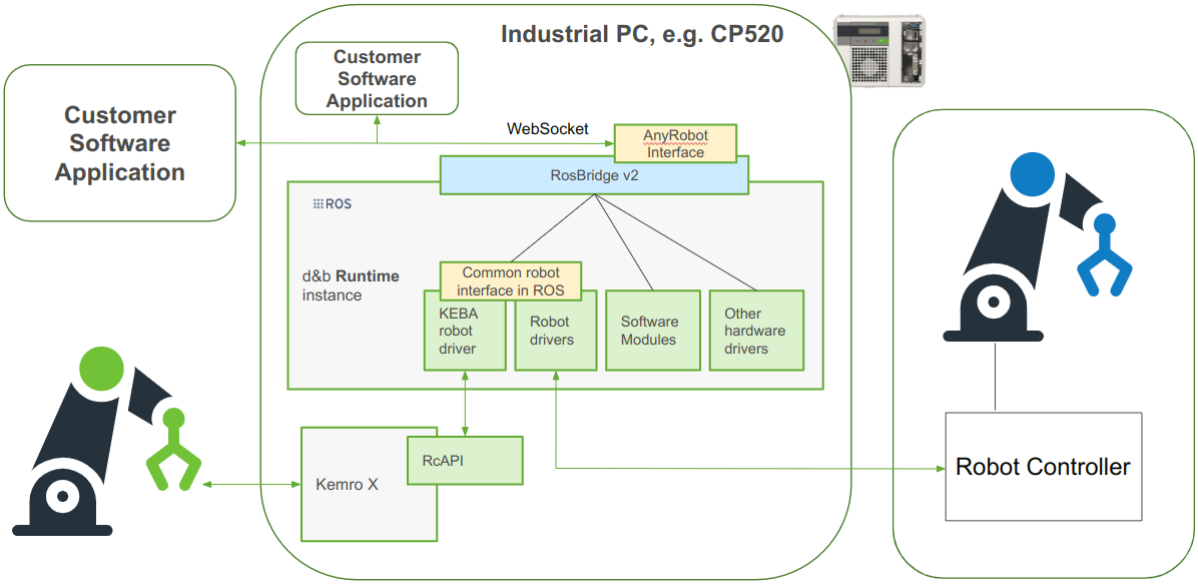


Figure 5.3: Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [11]).

On connection establishment, the WebSocket client performs the rosauth SHA-512 authentication handshake, because the drag and bot WebSocket port is accessible to any process on the local network and an unauthenticated endpoint would allow arbitrary motion command injection. The MAC is computed by hashing with SHA-512 concatenation of the shared runtime secret, the interface identifier, destination, a fixed nonce, session timestamps, and the access level string.

The authentication token is computed based on the procedure defined in the drag&bot AnyRobot Interface specification [11]:

$$\text{MAC} = \text{SHA-512}(\text{secret} \parallel \text{id_string} \parallel \text{dest} \parallel \text{nonce} \parallel \text{t} \parallel \text{level}) \tag{5.1}$$

SHA-512 is selected due to its strong cryptographic properties, as its 512-bit output makes pre-image and collision attacks computationally infeasible within the operational context of this application. The shared secret is stored as an environment variable to prevent exposure to version control systems. The concatenation order is strictly defined by the AnyRobot interface specification and must be followed precisely. Any deviation from this order results in a mismatch of the generated hash, causing the runtime system to reject the connection.

Listing 5.1 presents an illustrative Python implementation of the authentication token computation described in Equation 5.1, following the drag&bot AnyRobot Interface specification [11].

```python
import hashlib
import os

secret = os.getenv('RUNTIME_SECRET')

id_string = 'library'
dest = 'interface'
nonce = 'example_nonce'
timestamp = 0
role = 'user'

raw = secret + id_string + dest + nonce + str(timestamp) + role

mac = hashlib.sha512(raw.encode()).hexdigest()

auth_msg = {
    'operation': 'auth',
    'mac': mac,
    'nonce': nonce,
    'timestamp': timestamp,
    'role': role
}
```

Listing 5.1: Illustrative Python implementation of SHA-512-based authentication token computation following the drag&bot AnyRobot Interface specification.

Figure 5.4 shows the robot control panel used for system interaction and monitoring.

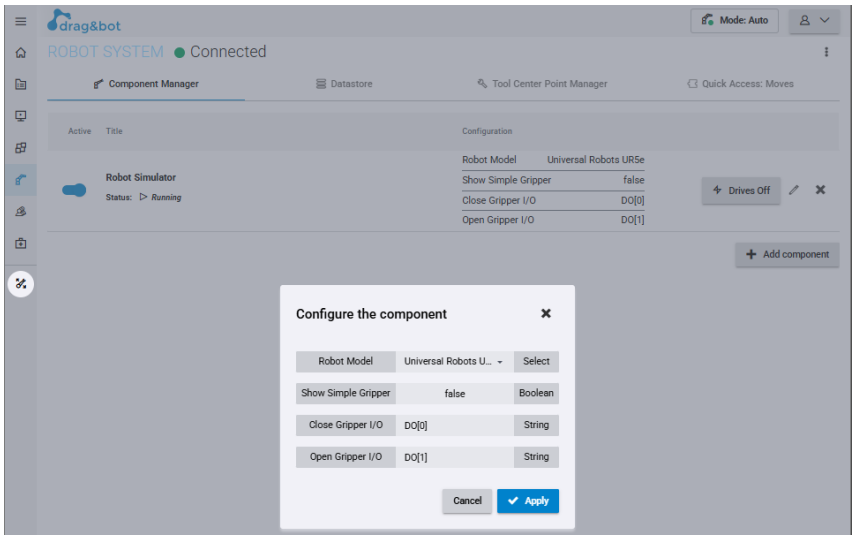


Figure 5.4: Robot control panel interface used for system operation and monitoring.

5.2.4 Configuration Architecture and YAML File Structure

The YAML configuration file serves as the central entry point for all process-related parameters and provides a flexible mechanism for adapting the system to different workpieces without requiring modifications to the underlying source code, thereby fulfilling NFR-02. At the beginning of each session, the file is parsed once, and the resulting configuration data is stored as a dictionary that is shared across all modules in Layers 2 to 4. The structure of the file is organized into six clearly defined top-level sections, each corresponding to a specific functional aspect of the system architecture, which supports clarity and maintainability. YAML is selected over JSON primarily due to its support for inline comments. This allows engineers to document the meaning and physical relevance of individual parameters directly within the configuration file, eliminating the need to consult external documentation. To ensure robustness and security, the configuration is loaded using a safe parsing method, which prevents the execution of unintended code in the event that the file is modified or corrupted. A key design decision concerns the handling of spatial consistency during the regrip process. Rather than defining approach and placement poses independently, all such poses are derived relative to a single reference pose defined for the orientation table. For example, the approach position is calculated by applying a fixed vertical offset to the placement pose. This design ensures that any change in the position of the orientation table requires only a single modification within the configuration file. In contrast, storing these poses independently could introduce inconsistencies: if the table position is updated but the corresponding approach position is not, the system may execute motions based on outdated spatial data. Such inconsistencies are particularly problematic because they may not be detected during configuration validation, as each parameter remains individually

valid despite being collectively incorrect.

Listing 5.2 presents a generalised YAML configuration illustrating the key process, motion, and safety parameters without exposing system-specific or confidential information.

```yaml
# =========================
# Robot configuration (generalised)
# =========================

robot:
  host: <robot_ip_address>
  port: <port_number>
  speed_mps: 0.5
  accel_percent: 20
  blend_percent: 15
  io_group: DO
  io_pin: 1

# =========================
# Process and sheet definition
# =========================

process:
  sheet:
    length_m: 0.4
    width_m: 0.3
    thickness_m: 0.005
    mass_kg: 2.5

  grip:
    cup_count: 2
    min_edge_clear_m: 0.05
    min_spacing_m: 0.2
```

Listing 5.2: Generalised YAML configuration illustrating robot parameters, process definition, and motion and safety constraints.

5.2.5 Operator interface – interactive 3d sheet bending viewer

Operator input for each execution cycle is provided through an interactive three-dimensional sheet bending viewer developed in this work (hereafter referred to as the *3D viewer*), rather than a conventional text-based interface. This design decision is primarily motivated by the spatial nature of bend side selection. By

presenting the workpiece directly as a 3D model, the operator can identify relevant surfaces and orientations more intuitively than by relying on abstract side labels. In practice, this reduces the likelihood of incorrect selections while keeping the interaction simple, as the operator performs only visually guided inputs and the system handles the underlying computations.

The 3D viewer is implemented as a web-based application using Plotly Dash and renders the workpiece as an interactive mesh derived from the corresponding STL file. It operates as a local service and is integrated into the drag&bot Operator Cockpit through the Custom URL panel mechanism. A new panel of type *Custom URL* is added to the dashboard via the dashboard configuration interface, where the local viewer address is specified as the target. Once configured, the viewer is rendered as an embedded panel within the cockpit, allowing the operator to interact with the 3D mesh and associated controls without leaving the drag&bot environment. The configuration is stored as part of the dashboard definition and remains available across sessions, ensuring a consistent setup for repeated operation.

This integration ensures that configuration, monitoring, and execution are performed within a single interface, eliminating unnecessary context switching and providing a more coherent workflow from the operator's perspective. The overall system setup used during development and validation, including the robot, press brake, and orientation table, is shown in Figure 5.5.

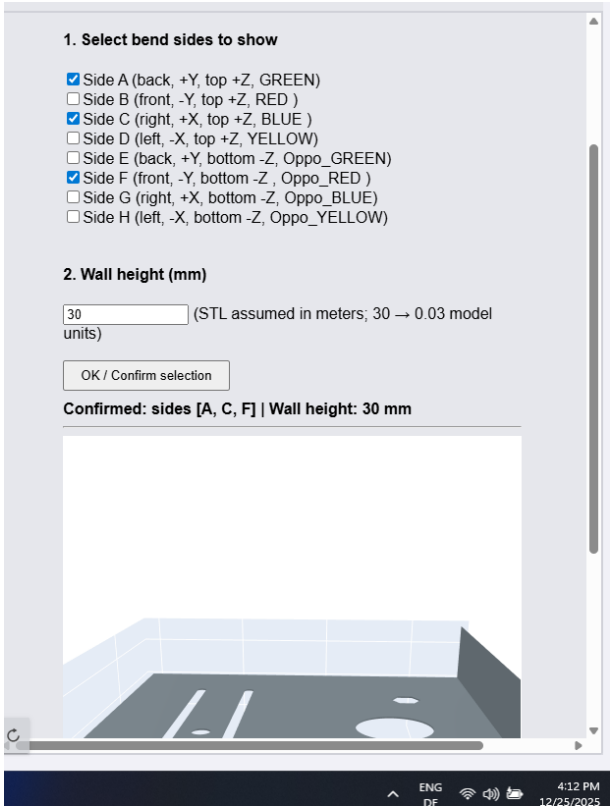


Figure 5.5: Virtual simulation environment used for implementation and validation. The setup includes the robot, press brake, orientation table, and sheet workpiece. Source: own work.

To accommodate different levels of operator expertise, the interface is divided into two modes. In basic mode, the operator selects from a predefined set of bending sides and specifies the required wall height. The system automatically prevents invalid combinations, particularly those that would require additional operations such as regripping, which are intentionally excluded at this level to simplify interaction. In contrast, expert mode exposes additional parameters, including geometric constraints, tool configuration, and pose definitions. However, changes made in this mode are not stored automatically and must be explicitly transferred to the configuration file, ensuring that persistent modifications are introduced in a controlled manner.

In parallel with user input, the system performs a geometric analysis of the imported mesh to identify internal features such as holes. These features are detected based on their orientation and their spatial relationship to the outer boundary of the sheet, and are subsequently classified according to their geometric characteristics. The resulting feature information is used during grip planning to exclude unsuitable regions from consideration. The workpiece model used for this analysis is illustrated in Figure 5.6. Additionally, the workpiece mass is estimated using a standard material density, providing a conservative reference for evaluating whether a given gripping configuration is physically feasible.

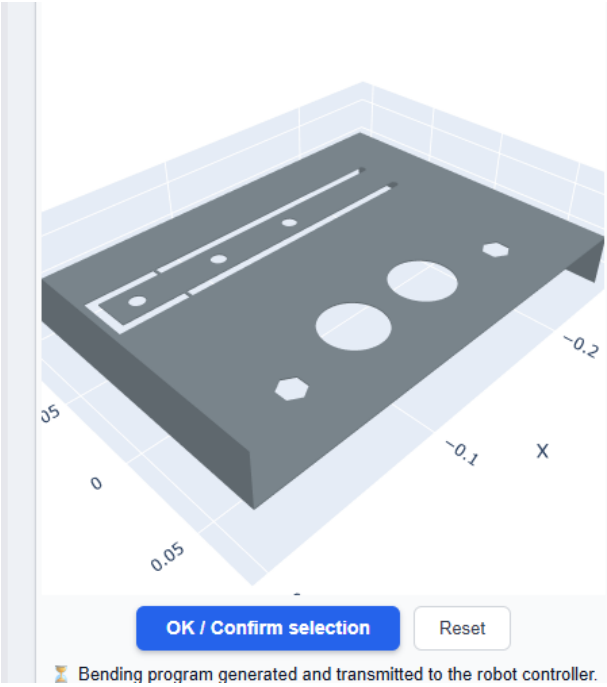
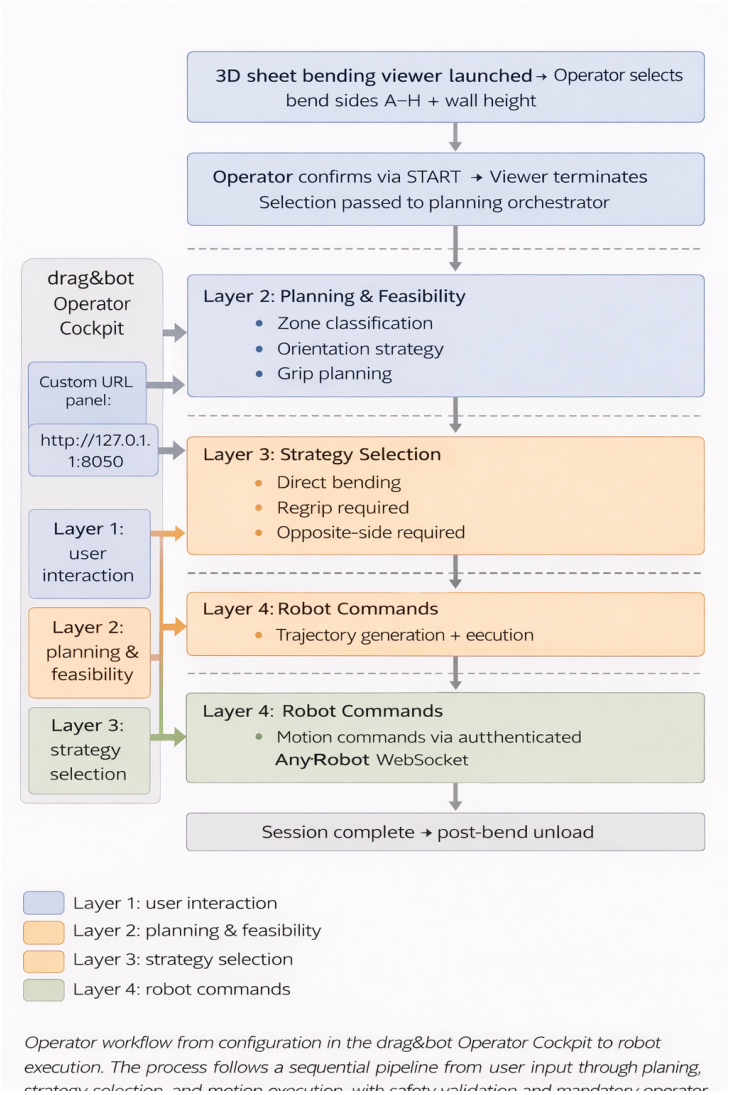


Figure 5.6: Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work.

Once the operator confirms the selection, the defined parameters are transmitted to the planning module in a structured format. The viewer then terminates after a short delay. This behaviour is intentional, as it prevents further interaction while

the robot is executing a trajectory, thereby avoiding inconsistencies that could arise if configuration changes were introduced during active motion.

The complete sequence of operator interaction and system processing is summarised in Figure 5.7, which illustrates the progression from initial configuration through planning, strategy selection, and final execution.



Operator workflow from configuration in the drag&bot Operator Cockpit to robot execution. The process follows a sequential pipeline from user input through planing,

Figure 5.7: Operator Workflow .Every gate must be passed before the next begins.All operator interactions occu within the drag and bot enviornment. *Source: own work.*

5.3 Development and simulation Environment

With the architectural foundation established in Section 5.1, the implementation requires a defined environment in which all modules can be executed, tested, and validated before involving any physical robot. This section outlines the environment, including the drag and bot virtual simulation platform, the UR5e robot model with its

TCP configuration, and the spatial zone classification used to divide the workspace and determine the execution path for each bending session.

5.3.1 Simulation Platform, Robot Model, and TCP Configuration

The drag and bot platform is used as the primary environment for development, integration, and testing within this thesis, as it combines a virtual robot controller and the AnyRobot WebSocket Interface within a single system, eliminating the need for additional simulation tools or direct access to physical hardware [11]. All Python-based modules are implemented and validated within this environment. The platform operates on the same Ubuntu system as the planning software, enabling communication through the local loopback interface and avoiding timing variations caused by external network conditions. The planning software establishes a single authenticated WebSocket connection to ws://<local-ip>:9091, which is used to handle all command types, including motion execution, IO operations, object manager interactions, and operator HI dialog requests. All session-related parameters are loaded from operator input during initialization and are explicitly passed to each module through function arguments. This approach ensures that dependencies remain transparent at the interface level and prevents modules from accessing outdated values from shared contexts. The same principle is consistently applied across all twenty-seven modules, with the configuration structure described in Section 5.2.4.

Within the simulation environment, the Universal Robots UR5e is selected as the manipulator model, as it corresponds to the robot used in the physical KEBA press brake system and is supported by a validated kinematic representation in the drag and bot platform [11]. The UR5e provides six degrees of freedom, a maximum reach of approximately 850 mm, and a payload capacity of 5 kg, which are sufficient for handling the sheet metal workpieces considered in this implementation [19]. All motion targets generated by the planning software are evaluated against this kinematic model, allowing reachability limitations to be identified during simulation rather than during operation on the physical system.

The Tool Centre Point applied to all motion targets is defined through the drag and bot tool management entry project_3_tcp, in accordance with SR-03 in Section 3.2.4 and based on a system requirement specified by the company. This ensures a consistent reference for tool geometry so that all generated poses correspond to the actual gripper configuration. The TCP is defined using the ZYX Euler convention with the following offsets relative to the robot flange frame: x = 0.000 m, y = 0.034 m, z = 0.194 m, RZ = −90.0°, RY = −75.0°, RX = 0.0°. The Z offset of 194 mm represents the distance from the flange to the suction contact surface, while the Y offset of 34 mm reflects the asymmetric arrangement of the suction cups along the gripper. All coordinates computed by the planning software are defined at this TCP, allowing the drag and bot runtime to derive the corresponding flange pose

through inverse transformation without requiring the planning layer to explicitly consider flange geometry. The TCP identifier project_3_tcp is stored in operator input, enabling flexible adaptation of the system. If adjustments are required in the future, the TCP can be updated by the user through parameter changes in the operator interface, without requiring modifications to the source code.

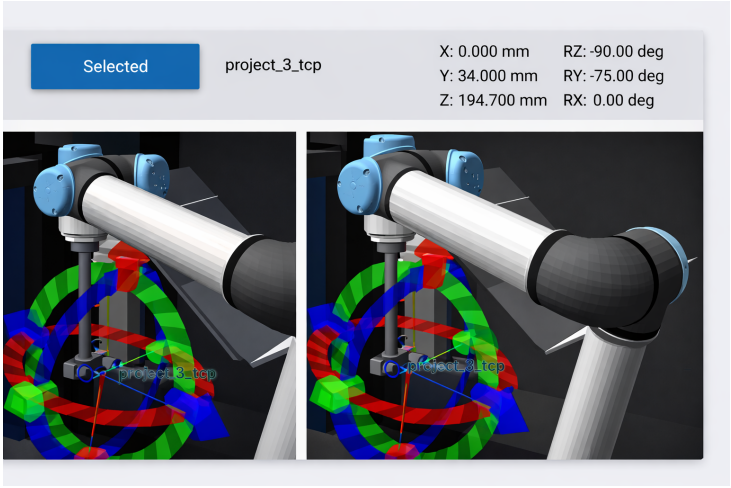


Figure 5.8: TCP of UR5e robot. *Source: own work.*

5.3.2 Virtual Cell Layout and Spatial Zone Classification

The drag and bot simulation cell includes the UR5e robot, the press brake model, the orientation table, and a sheet metal workpiece whose tracked pose (primary tracked sheet object) forms the basis for all geometric calculations in the planning software. In addition, a secondary sheet object is registered to support two-holder operation, allowing the system to track whichever workpiece is active during execution. All fixture positions, workspace limits, and zone thresholds defined in operator input module, which is aligned with the physical cell layout, ensuring that the simulation results correspond directly to the real system configuration.

At the start of each bending session, the spatial zone classifier retrieves the six-degree-of-freedom pose of the sheet from the object tracking service in the robot world coordinate frame. The origin of this frame is located at the robot base, with the X axis defined laterally and the Y axis longitudinally across the cell. Based on this reference, the XY position of the sheet centroid is evaluated using a threshold-based rule to assign the workpiece to one of four spatial zones. This process is performed automatically using live tracking data without operator input. The classification is defined relative to the robot base position, which must be correctly specified in the operator input configuration.

The XY plane is divided into four regions around the robot base. A centroid with negative X and positive Y is assigned to Zone 1 (rear-left), while negative X and negative Y correspond to Zone 2 (front-left). Positive X combined with posi-

tive Y defines Zone 3 (rear-right), and positive X with negative Y defines Zone 4 (front-right). The classification thresholds are set to ±10 mm in the X direction and ±50 mm in the Y direction. These asymmetric limits reflect the physical cell geometry and account for increased tracking uncertainty along the longitudinal axis [1]. Zones 3 and 4 lie within the primary reachable workspace of the UR5e and therefore allow direct execution of the pickup and bending sequence from the nominal home configuration. In contrast, Zones 1 and 2 are in a region with limited reachability, requiring an automatic repositioning phase. In this phase, the robot moves to an automatically calculated predefined intermediate configuration within the reachable workspace, after which the standard bending sequence is executed. This repositioning is triggered by the classified zone and is performed without operator intervention. A subsequent tracking update verifies that the sheet lies within the reachable region before execution continues.

If the sheet centroid lies within the dead-band region defined by $|x| < 10$ mm or $|y| < 50$ mm, the classifier returns to an unresolved state. This occurs near the coordinate axes, where small tracking variations may lead to inconsistent zone assignment. To prevent incorrect execution, the system terminates the session and issues blocking notification instructing the operator to reposition the workpiece before restarting.

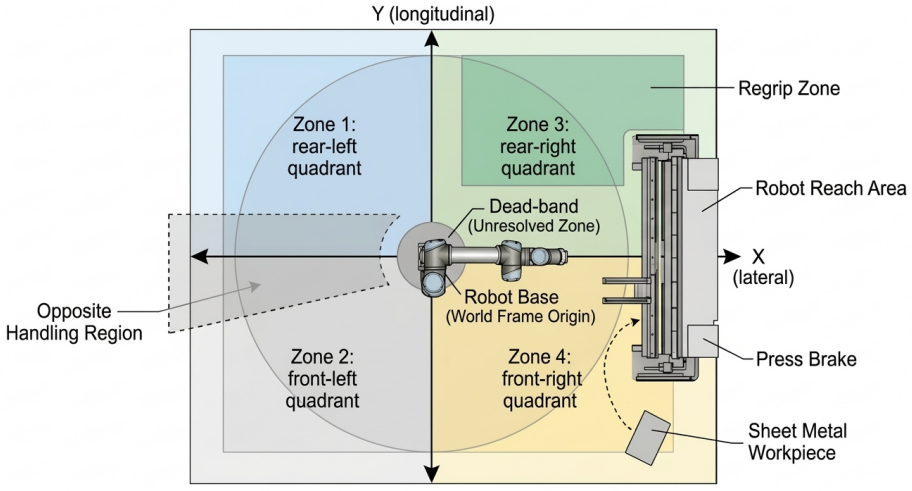


Figure 5.9: Zone classification for robot workspace segmentation. *Source: own work.*

Having defined the simulation environment, robot model, TCP configuration, and zone classification, the following sections describe the functional subsystems in the execution order of a bending session, starting with safety validation, followed by orientation determination, grip planning, and trajectory generation.

5.4 Pre-Execution Safety Validation

Safety validation is carried out before any motion is generated to ensure that unsafe conditions and infeasible configurations are identified prior to robot execution, in accordance with NFR-03, NFR-04, and NFR-05. The system incorporates three complementary mechanisms to achieve this: a fixture poses deviation check, a blocking operator confirmation dialog, and a runtime command error watcher. In addition, all trajectory waypoints are generated automatically and verified using robot kinematics to confirm reachability and feasibility before execution, preventing invalid motion commands from being issued.

5.4.1 Fixture Pose Deviation Validation

The fixture pose deviation check retrieves the current poses of six critical fixtures from the drag&bot object manager and compares them with reference poses obtained during the initial calibration of the cell. The monitored elements include the primary sheet object at the press brake, the finished-part position on the regrip table base and support, and the placement and attachment references at the flip station. These fixtures are treated as fixed references within the planning software, since all grip positions, trajectory waypoints, and placement operations are computed under the assumption that their positions remain unchanged. Any deviation from these reference poses can therefore result in motion targets that are geometrically valid but physically incorrect.

The deviation check is applied uniformly across all spatial zones. Each fixture is evaluated using both positional and orientation criteria. The positional deviation is computed as the Euclidean distance

$$\Delta_{pos} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{5.2}$$

and the orientation deviation is defined as

$$\Delta_{ang} = \max\left(|\Delta RZ|, |\Delta RY|, |\Delta RX|\right). \tag{5.3}$$

A fixture is considered out of tolerance when Δ_{pos} exceeds 5 mm or Δ_{ang} exceeds 0.10 rad (5.7°). The positional threshold is selected to remain below the minimum clearance used in grip planning, ensuring that fixture misalignment does not produce misleadingly valid grip configurations. The angular threshold corresponds to the minimum deviation that would affect the accuracy of the vacuum cup contact on the sheet surface. The primary sheet object is evaluated using the same deviation criteria as all other fixtures. Since the planning software relies on the current sheet pose for all geometric computations, any displacement from the expected reference may lead to incorrect motion generation. Therefore, the sheet is included in the deviation evaluation in the same manner as other fixtures, and any detected deviation

is treated as a potential configuration error. If one or more fixtures exceed the defined thresholds, the system generates a structured warning that lists all affected fixtures using their generic labels, together with the measured positional and angular deviations. This warning is presented through a blocking HI dialog titled *Fixture position warning*, which allows the operator to either continue with explicit caution or cancel the operation for further inspection. The operator's decision is recorded, as the responsibility for accepting out-of-tolerance conditions remains with the human operator. If a fixture is not detected in the object manager response, it is treated as missing and is automatically flagged regardless of zone classification.

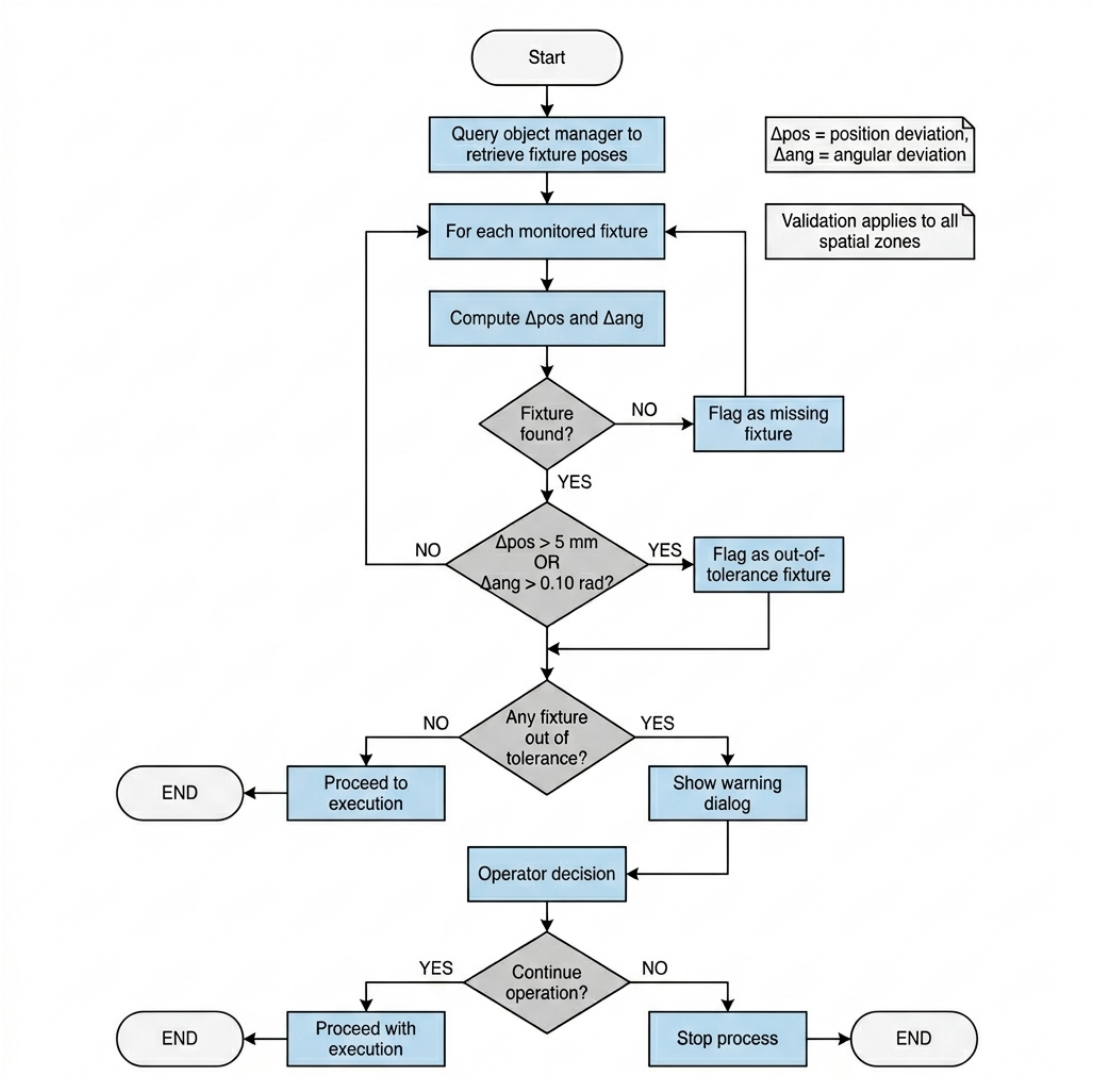


Figure 5.10: Fixture pose deviation validation workflow. Each monitored fixture is evaluated by comparing positional and angular deviations against defined thresholds. Out-of-tolerance or missing fixtures trigger a blocking warning dialog requiring operator confirmation. Validation is applied uniformly across all spatial zones. Source: own work.

5.4.2 Operator Authorization and Runtime Error Monitoring

The operator confirmation dialog is positioned as the first step in the planning orchestrator startup sequence, preceding all subsequent processes including fixture validation, visualization, and motion command transmission. The dialog enforces a mandatory decision: either initiate the automated workflow or cancel execution. This behavior is structurally enforced, as the orchestrator does not proceed until a response is received from the HI service. Consequently, execution cannot begin without explicit operator authorisation, ensuring compliance with safety requirements. If the operator selects cancel, the orchestrator terminates without issuing any command to the robot runtime.

In parallel with this confirmation step and all subsequent execution phases, a motion error watcher operates as a background subprocess. This component subscribes to the `/command_result` ROS topic through an independent WebSocket connection, enabling continuous monitoring of command execution outcomes. A separate connection is required to maintain an uninterrupted receive loop without blocking the orchestrator's command flow. When a non-zero result code is detected, indicating a motion execution failure, the watcher immediately triggers a modal HI notification to inform the operator. This ensures that execution errors are identified and communicated in real time.

The error watcher does not directly interrupt or terminate the orchestrator. Instead, it provides immediate feedback through the HI interface, requiring operator acknowledgement before further interaction is permitted. This design ensures that execution anomalies do not pass unnoticed, while maintaining operator authority over subsequent actions.

Together with the fixture pose deviation check, these mechanisms establish a layered safety structure. The confirmation dialog enforces operator authorization prior to execution, while the error watcher ensures that failures during execution are promptly detected and communicated. Both mechanisms utilize the drag&bot HI service, providing a consistent interface for all safety-related interactions. With these safeguards in place, the subsequent section addresses the workpiece orientation strategy for the bending process.

5.5 Workpiece Orientation Strategy

Following pre-execution safety validation, the first geometric step in each bending cycle is the determination of a valid pickup orientation for the sheet. This is necessary because the sheet may be placed in arbitrary orientations within the workspace, while only a subset of these orientations is kinematically feasible for the target bending operation. The orientation strategy evaluates the tracked sheet pose, applies a structured selection process, and produces a validated pickup pose for subsequent grip planning.

5.5.1 Zone-Based RZ Admissibility

The admissible range of the sheet rotation about the Z-axis is defined by a zone-dependent window $[RZ_{\text{min}}, RZ_{\text{max}}]$. This constraint reflects the limited rotational capability of the robot end-effector within different regions of the workspace. Since the reachable orientation range varies with the spatial position of the sheet, zone classification is performed prior to orientation evaluation. The corresponding admissibility window is retrieved from the configuration, ensuring that only feasible orientations are considered. This approach avoids selecting poses that appear valid geometrically but are unreachable during execution.

5.5.2 Pickup Orientation Validation

Before applying the orientation selection strategy, the tracked pose is validated to ensure that the gripper approaches the sheet from above. This is evaluated using the product $\cos(RX) \cdot \cos(RY)$. If the value is below -0.10, the tool Z-axis is oriented downward and no modification is required. Otherwise, a corrective transformation is applied:

$$RZ' = \text{wrap}(RZ + 180^\circ), \quad RY' = -RY, \quad RX' = \text{wrap}(RX + 180^\circ) \tag{5.4}$$

This transformation ensures that the gripper is correctly aligned with the sheet surface without altering the pickup position.

5.5.3 Three-Level Orientation Selection Strategy

To ensure that a valid pickup orientation can be determined for any initial placement, a three-level selection strategy is employed. The strategy is evaluated sequentially in a strict priority order, and execution terminates as soon as a valid orientation satisfying the admissibility constraints is identified. This approach guarantees robustness against arbitrary initial sheet orientations while maintaining preference for geometrically consistent and kinematically favorable configurations.

The workpiece is defined with four reference faces established during system commissioning, each associated with a fixed angular offset about the sheet's local vertical axis. These faces provide a structured reference for interpreting the tracked orientation. The offsets correspond to 0°, 180°, $+90^\circ$, and -90° relative to the reference face. By compensating for these offsets, the system can normalise any detected orientation into a common reference frame, enabling consistent evaluation across different initial conditions.

Level 1: Reference-Face Equivalent Orientation At the highest priority level, the tracked orientation is converted into its reference-face equivalent by re-

moving the face-specific angular offset. This is achieved by applying an inverse rotation about the sheet's local Z-axis:

$$R_{\text{new}} = R_{\text{raw}} \cdot R_z(-d) \tag{5.5}$$

where d represents the angular offset associated with the detected face. The resulting rotation matrix is then decomposed into ZYX Euler angles using:

$$R = R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma) \tag{5.6}$$

$$RZ = \text{atan2}(R_{10}, R_{00}), \quad RY = \text{arcsin}(-R_{20}), \quad RX = \text{atan2}(R_{21}, R_{22}) \tag{5.7}$$

The computed RZ value is then evaluated against the zone-specific admissibility window $[RZ_{\text{min}}, RZ_{\text{max}}]$. If the orientation lies within this range, it is selected as the pickup pose. This level is preferred because it aligns all configurations to a common reference frame, resulting in more predictable motion generation and reduced trajectory complexity.

Level 2: Detected-Pose Evaluation If the reference-face equivalent orientation falls outside the admissible range, the strategy proceeds to evaluate the raw tracked orientation directly. This level accounts for cases where the sheet is already positioned within a valid orientation relative to the robot workspace, and the reference-face conversion would unnecessarily shift it outside the admissible window.

The raw RZ value is therefore checked against the same admissibility limits. If it satisfies the constraint, the raw pose is accepted as the pickup orientation. This level preserves valid initial configurations without enforcing additional transformations.

Level 3: Inverted-Pose Fallback If both the reference-face equivalent and the raw orientation fail to satisfy the admissibility condition, a fallback orientation is generated by applying a 180° inversion:

$$RZ' = \text{wrap}(RZ \pm 180^\circ), \quad RY' = -RY, \quad RX' = -RX \tag{5.8}$$

This transformation effectively rotates the sheet about its vertical axis while reversing its orientation about the horizontal axes. The inverted pose expands the set of reachable configurations and ensures that a pickup orientation can always be produced, even when the initial pose lies outside the calibrated admissible region.

However, since this orientation may not strictly satisfy the predefined admissibility constraints, it is marked with an *admissibility unconfirmed* flag. This flag is propagated to downstream modules, including the grip planner and trajectory

generator, allowing them to apply additional safety margins or conservative motion constraints when executing the operation.

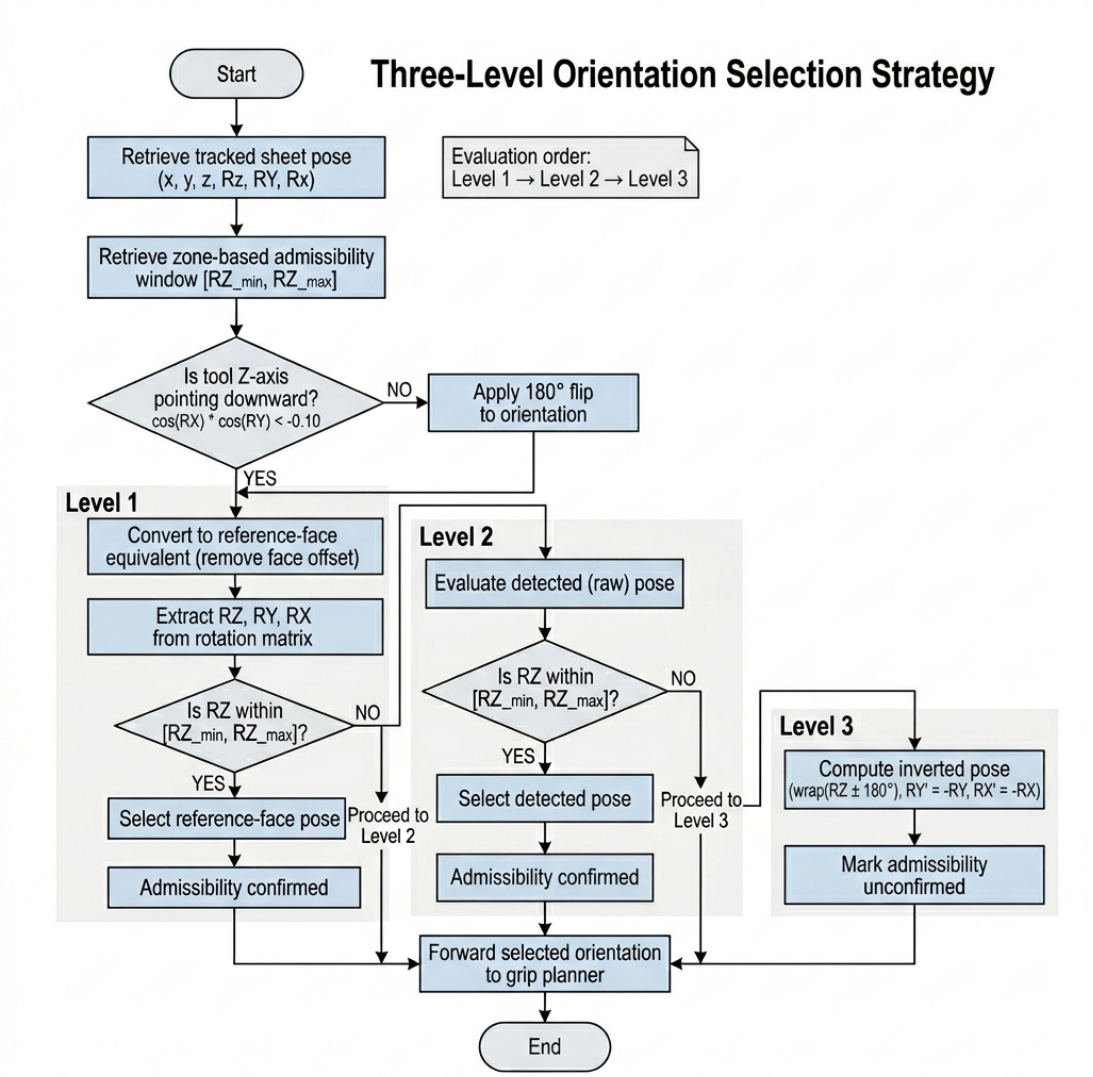


Figure 5.11: Three-level system architecture illustrating the hierarchical structure of the implementation. *Source: own work.*

Strategy Outcome The three-level strategy guarantees that a pickup orientation is always determined while prioritising configurations that are both kinematically feasible and geometrically consistent. By enforcing a strict evaluation order and incorporating a controlled fallback mechanism, the approach balances robustness and operational safety. The final selected orientation, together with its admissibility status, is forwarded to subsequent planning stages for execution.

Bibliography

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, 2nd ed. London: Springer, 2016, ISBN: 978-1-4471-1500-0.

[2] S. Aomura and K. Koguchi, "Development of a robotic bending system for sheet metal," *Journal of Materials Processing Technology*, vol. 130–131, pp. 362–367, 2002. DOI: 10.1016/S0924-0136(02)00744-1

[3] Z. Pan, J. Polden, N. Larkin, S. van Duin, and J. Norrish, "Recent progress on programming methods for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 20, no. 2, pp. 137–150, 2012. DOI: 10.1016/j.rcim.2003.07.004

[4] J. Krüger, T. K. Lien, and A. Verl, "Cooperation of human and machines in assembly lines," *CIRP Annals*, vol. 58, no. 2, pp. 628–646, 2009. DOI: 10.1016/j.cirp.2009.09.009

[5] C. Schou, J. S. Damgaard, S. Bøgh, and O. Madsen, "Human–robot interface for instructing industrial robots using skill primitives," *Robotics and Computer-Integrated Manufacturing*, vol. 29, no. 6, pp. 403–412, 2013. DOI: 10.1016/j.rcim.2013.06.002

[6] A. Björkelund, J. Malec, K. Nilsson, and P. Nugues, "Knowledge-based industrial robot task planning and execution," *Robotics and Computer-Integrated Manufacturing*, vol. 28, no. 5, pp. 558–568, 2012. DOI: 10.1016/j.rcim.2012.01.003

[7] S. K. Gupta, D. A. Bourne, K. Kim, and S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998. DOI: 10.1016/S0278-6125(98)80035-2

[8] Q. Glorieux, J. Vierendeels, and J. De Schutter, "Quality- and productivity-driven trajectory optimisation for robotic handling," *Robotics and Computer-Integrated Manufacturing*, vol. 54, pp. 37–49, 2018. DOI: 10.1016/j.rcim.2018.05.001

[9] X. Liao and G. G. Wang, "Evolutionary path planning for robot-assisted sheet metal bending," *Journal of Manufacturing Systems*, vol. 22, no. 2, pp. 112–127, 2003. DOI: 10.1016/S0278-6125(03)80006-4

[10] J. R. Duflou, D. Van Oudheusden, J.-P. Kruth, and D. Cattrysse, "Modeling of vacuum grippers for the design of energy-efficient vacuum-based handling processes," *CIRP Annals*, vol. 55, no. 1, pp. 17–20, 2006. DOI: 10.1016/S0007-8506(07)60375-0

[11] drag&bot GmbH. "Anyrobot interface documentation." Ulm, Germany, Accessed: Mar. 2, 2026. [Online]. Available: https://docs.dragandbot.com/anyrobot-interface

[12] K. Thramboulidis, "Model-driven engineering for industrial automation systems: A survey," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 1893–1903, 2013. DOI: 10.1109/TII.2013.2251233

[13] Wikipedia contributors, *Euler angles — wikipedia, the free encyclopedia*, [Online; accessed 9-March-2026], 2026. [Online]. Available: https://en.wikipedia.org/wiki/Euler_angles

[14] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005, ISBN: 978-0-13-123629-5.

[15] M. Bär, G. Reinhart, and J. Schilp, "Virtual commissioning of robotic systems: A review and outlook," *Procedia CIRP*, vol. 72, pp. 129–134, 2018.

[16] G. Pahl, W. Beitz, J. Feldhusen, and K. H. Grote, *Engineering Design: A Systematic Approach*, 3rd ed. London: Springer, 2007. DOI: 10.1007/978-1-84628-319-2

[17] M. Quigley et al., "ROS: an Open-Source Robot Operating System," in *Proceedings of the ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[18] Python Software Foundation, *Python 3 language reference*, Accessed: 2026-03-02, 2024. [Online]. Available: https://docs.python.org/3/

[19] Universal Robots A/S, *UR5e Technical Specifications*, https://www.universal-robots.com/products/ur5-robot/technical-specifications/, Accessed: Mar. 2026, Odense, Denmark, 2026.